Rapport de projet de fin d'études

Présenté par :
Ali NCIBI

Pour l'obtention du :
Diplôme d'ingénieur

Repérage des liens temporels rencontrés sur les textes français par des méthodes de deep-learning

Réalisé à :

CESP U1018

Du 01/03/2021 au 31/08/2021

Superviseur INSERM	**:**	**Mme**	**Marie-Hélène METZGER**
Tuteur de l'école	**:**	**Mme**	**Laurence LIKFORMAN**

Année universitaire : 2020/2021

Table des matières

Table des figures 2

Introduction 3

1. Etude bibliographique 4

1.1 Approche manuelle des linguistes pour l'étude des liens temporels 4

1.2 Méthodes classiques de repérage des temporalités par apprentissage supervisé 4

1.3 Méthodes basées sur le deep-learning et des annotations temporelles « allégées » 5

1.3.1 La reconnaissance des objets temporels 5

1.3.2 Election, et classification des paires selon la nature de la temporalité 7

2. Méthodologie 8

2.1 Source de données annotées : FrenchTimeBank (FTiB) 8

2.1.1 Répartition disjonctive des classes temporelles 8

2.1.2 Expression simplifiée d'un lien temporel 9

2.2 Formalisation de la tâche 10

2.3 Architecture du système 12

2.3.1 Couches externes d'embedding 12

2.3.2 Couche spécifique d'embedding 12

2.3.3 Concaténation des différentes couches d'embedding 12

2.3.4 (bi)LSTM-CRF pour la reconnaissance des entités temporelles 13

2.3.5 Vérification des liens temporels par classification des paires candidates 13

2.4 Schémas d'entrainement 15

2.4.1 Approche par apprentissages séparés 15

2.4.2 Approche multi-task-learning 16

2.5 Classification multiple ou binaire 16

2.6 Métriques d'évaluation 17

2.7 Ré-échantillonnage des exemples annotées de (FTiB) 18

3. Expériences 20

3.1 Construction des ensembles d'apprentissage 20

3.2 Résultats d'apprentissage 21

Conclusion 22

Perspectives 22

Annexe 23

Références 25

Table des figures

Figure 01 - Exemple de graphe établi par dependancy-parsing d'un bout de texte. 4
Figure 02 - Déroulement en deux phases de la tâche de repérage et de classification des temporalités. 5
Figure 03 - Exemple d'un texte annoté par rapport au temps suivant la norme ISO-TimeML. 6
Figure 04 - Schéma montrant le passage d'un TLINK à une définition plus simplifiée du lien temporel. 9
Figure 05 - Reconnaissance des entités temporelles par un module étiqueteur de séquences. 10
Figure 06 - Mécanisme d'élection - exhaustive - des paires candidates distinctes. Ici $Qs = 3$. 11
Figure 07 - Classification des paires temporelles candidates. 11
Figure 08 - Projection d'une séquence de tokens via les couches d'embedding et concaténation des features. 12
Figure 09 - Etiquetage des tokens temporels par un module (bi)LSTM-CRF. 13
Figure 10 - Module de classification des paires candidates selon la nature exacte de la temporalité. 14
Figure 11 - Mécanisme de ré-échantillonnage des documents annotés de (FTiB) 19
Figure 12 - Nombre d'échantillons d'apprentissage extraits par classe temporelle retenue. 20

Introduction

L'objectif de ce travail est de résoudre les problèmes de repérage des entités temporelles rencontrées dans les textes français, et de classification des éventuelles relations entre elles.

Afin d'atteindre cet objectif combiné, nous allons proposer conformément aux travaux similaires récents un système automatisé à deux phases basé dans son fonctionnement sur des architectures adaptées du deep-learning, qui va résoudre le problème en le divisant en deux sous-tâches :

Une première sous-tâche de reconnaissance des entités temporelles qui seraient impliquées dans certaines relations de classes connues, modélisée par un module étiqueteur de séquences, et une deuxième phase de classification de certaines paires candidates d'objets temporels parmi ceux précédemment identifiés, en fonction de la nature (classe) exacte des liens observés (précédence, simultanéité, inclusion, etc.), modélisée par un autre module classificateur.

L'automatisation de la phase de classification des paires candidates nécessiterait d'une part le choix d'une stratégie bien adaptée d'élection de certaines paires candidates, et d'autre part le choix d'une méthode d'extraction de features qui caractériseraient ces paires à l'entrée d'un classificateur.

Dans les approches classiques, de telles représentations (features) sont construites totalement ou partiellement à la main, généralement en se référant à des règles de décisions de natures morphosyntaxiques spécifiques à la nature de la relation étudiée. Nous souhaitons à l'occasion de ce travail éviter ces constructions coûteuses des features, et nous allons essayer d'automatiser le processus de génération de celles-ci simplement en concaténant les représentations issues par projections sur certaines couches d'embedding convenablement entrainées et importées dans notre système. Nous chercherons à voir globalement à quels points de telles features génériques apprises automatiquement seraient-elles efficaces et pertinentes pour la résolution des deux sous-tâches envisagées, en confrontant plusieurs techniques possibles d'embedding.

Pour entrainer les deux composantes d'un tel système, en mode supervisé, nous allons nous baser sur les documents annotés de FrenchTimeBank (FTiB), le corpus de référence pour l'étude des temporalités en français. Les étiquettes temporelles qui y sont exprimées ont été établies selon le schéma d'annotation ISO-TimeML et sont souvent qualifiées de « surfaciques » car elles sont plus simples à établir et à interpréter que les annotations grammaticales classiques des informations temporelles. Cette légèreté dans la façon d'encoder l'information sur les temporalités, instaurée à l'occasion de tels schémas, devrait simplifier significativement la construction et l'exploitation de corpus d'apprentissage d'où l'intérêt à ce type d'annotations dans notre travail.

Les deux sous-tâches impliquées dans notre modélisation seront en fait envisagées comme étant des sous-tâches de classification : d'abord l'étiquetage (la classification) de chaque token du texte, et puis la classification des paires candidates. Il convient alors de contrôler les performances des différentes composantes avec des matrices de confusion convenablement définies qui vont nous permettre de bien tracer la justesse de chaque sous-tâche.

Dans la partie expérimentale de ce projet, nous allons définir et utiliser de telles métriques pour expérimenter sur certaines classes extraites de (FTiB) et conclure quant à l'efficacité des features génériques telles que construites dans nos modèles à la résolution du problème traité.

Le volume très limité de documents initialement annotés par rapport aux temporalités dans (FTiB) ne nous permettra pas de généraliser. Cependant, nous allons essayer de démontrer, sous ces conditions spécifiques d'entrainement, qu'avec des annotations allégées, des architectures profondes adaptées, et des features génériques, il est possible d'automatiser la résolution de la tâche en question, sans beaucoup d'efforts comme dans les approches de machine-learning classiques.

1. Etude bibliographique

1.1 Approche manuelle des linguistes pour l'étude des liens temporels

Pour le repérage et l'étude d'une relation spécifique entre les parties d'un texte donné, les linguistes commencent typiquement par découper le texte en entités (tokens), et ils construisent ensuite ce qu'on appelle des arbres de dépendances entre celles-ci (Kolomiyets and Moens, 2011, p. 5413) en se basant sur des règles théoriques adaptées à la nature des relations étudiées : causalité, temporalité, etc. En l'occurrence, pour l'étude des temporalités le repérage et la caractérisation d'un lien donné entre deux parties du texte repose essentiellement sur l'établissement et l'analyse du shortest-dependancy-path (SDP) entre ces entités (Tang et al., 2013, p. 831). En effet, sur un graphe préétabli de dépendances, généralement de natures morphosyntaxiques, le chemin de liaison entre deux entités données du texte sera d'abord extrait, puis il sera analysé selon certaines règles grammaticales relativement sophistiquées, et les éventuelles relations temporelles entre les entités qui doivent y être exprimées seront ensuite repérées, requérant une certaine expertise de la personne qui résout la tâche. (Kolomiyets and Moens, 2011, pp. 5419–5420)

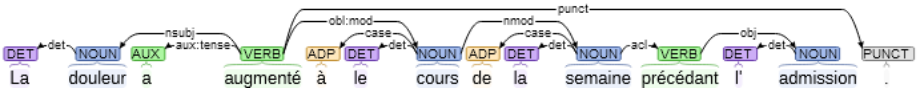

Figure 01 - Exemple de graphe établi par dependancy-parsing d'un bout de texte.

Ce processus de repérage et d'analyse « manuelles » des temporalités est difficile à réaliser. Il requiert en fait une analyse manuelle soigneuse des dépendances préétablies entre les parties du texte étudié, et il exige de se référer à de règles de décisions théoriques qui nécessitent un niveau élevé de connaissance dans le domaine duquel relève le texte afin de bien répondre à des questions sur les temporalités qui y sont exprimées. De nombreux travaux de recherches se sont donc orientés à l'automatisation progressive des méthodes de résolution de la tâche d'étude des temporalités rencontrés sur les textes écrits en langue naturelle, en veillant principalement à réduire les difficultés de résolution mentionnées ci-haut.

1.2 Méthodes classiques de repérage des temporalités par apprentissage supervisé

Une large famille de méthodes de machine-learning a été proposée dans la littérature à cet effet couvrant plusieurs domaines d'applications, notamment les applications cliniques (Alfattni et al., 2020, pp. 4–7) où des contextes narratifs sont très souvent rencontrés dans les textes des comptes rendus médicaux des patients. Les toutes premières continuaient de s'appuyer sur le fait que le (SDP) entre deux entités du texte doit être capable de capter l'information nécessaire pouvant ensuite renseigner sur les potentielles interactions - temporelles - entre elles. L'établissement de ces dépendances sert principalement dans le cadre de ces méthodes à l'ingénierie « manuelle » de features numériques censées être capables plus tard de bien incorporer de l'information sur les éventuelles interactions temporelles entre les entités qu'elles représentent (Tang et al., 2013, p. 832). Ces algorithmes se déroulent généralement en deux phases :

Une première phase d'identification des entités (mot, ou groupe de mots adjacents) qui seraient pertinentes au sens de la relation étudiée (Mirza and Tonelli, 2016, p. 69) : un modèle de machine learning est typiquement entrainé à l'occasion de cette phase, souvent envisagée comme sous-tâche d'étiquetage de texte suivant un schéma d'apprentissage supervisé par étiquetage grammatical. Son rôle est de prédire certaines étiquettes (part-of-speech tags par exemple) qui devraient aider à exprimer des propriétés de dépendances grammaticales entre les entités d'intérêt, et qui les distingueraient des autres parties du texte étudié (Jason et al., 2017, p. 2; Mirza and Tonelli, 2016, pp. 70–71). Et une deuxième phase qui commence par le calcul d'une certaine représentation numérique pour des paires candidates d'objets temporels parmi ceux précédemment identifiés, obtenue généralement après une analyse et un encodage des chemins de dépendances correspondants, suivi d'une classification de ces paires selon l'existence ou non d'une relation donnée, et ce après avoir observé leurs représentations numériques associées calculées au préalable. La classification sera effectuée d'une manière semi-autonome (extraction « manuelle » des features) en mode production, grâce à un entrainement supervisé d'un module classificateur sur des données convenablement préparées (Cheng and Miyao, 2018, pp. 1834–1835; Leeuwenberg and Moens, 2017, p. 1153).

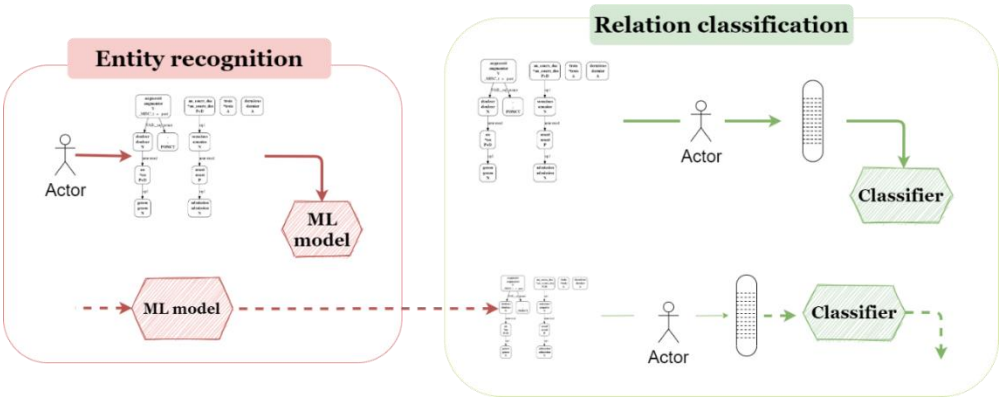

Figure 02 - Déroulement en deux phases de la tâche de repérage et de classification des temporalités.

Sans grande différence, les approches qui ont suit adoptaient pratiquement le même schéma de fonctionnement avec plutôt pour la partie classification un large panel de choix pour les représentations numériques (features) des paires candidates (Alfattni et al., 2020, p. 107; Mirza and Tonelli, 2016, p. 67; Tang et al., 2013, p. 832), autres que celles déduites exclusivement par analyse des chemins de dépendances (Leeuwenberg and Moens, 2017, pp. 1152–1153). Plusieurs applications cliniques faiblement supervisées continuent d'explorer les relations dans les textes en utilisant des règles spécifiques à la tâche, ou encore en employant une analyse syntaxique des dépendances et les grammaires compositionnelles, pour automatiser la classification des relations et normaliser les concepts cliniques (Callahan et al., 2019, p. 94; Peterson et al., 2020, p. 3).

Dans la pratique l'ingénierie manuelle de features significatives demeure une tâche relativement difficile et coûteuse : Quels attributs devrions-nous choisir pour représenter les entités afin de mieux intégrer l'information spécifique sur les relations entre elles ? Quelles pertes encourons-nous en faisant l'hypothèse de travailler avec tels ou tels attributs ?

D'ailleurs, le niveau d'automatisation atteint dans ces méthodes reste très limité, et il ne réduit pas trop les exigences de l'approche purement manuelle des linguistes en termes d'effort et d'expertise, ni dans le développement de l'algorithme ; car l'entrainement des modèles nécessiterait toujours une annotation très délicate d'un volume important de données, ni durant son utilisation en production ; ingénierie manuelle des features avant de prédire la classe de la relation.

Par ailleurs, dans ces circonstances, c'est-à-dire ayant accès aux graphes de dépendances entre les parties du texte, et ayant connaissance des règles de décision, un algorithme déterministe serait capable de résoudre la tâche en question par recherche « brutales » dans toutes les règles (déterministes) qui auraient été impliquées dans les décisions des experts linguistes, ce qui remet en question le recours à des modèles de machine-learning et introduire dès lors une incertitude dans les futures décisions. Ceci demeure un inconvénient majeur pour cette famille de méthodes même si l'idée derrière l'emploi de ces modèles était de réduire l'effort en automatisant partiellement le processus.

1.3 Méthodes basées sur le deep-learning et des annotations temporelles « allégées »

De nos jours, et grâce à la disponibilité accrue de corpus annotés relativement aux temporalités (Bittar et al., 2011a; Pustejovsky et al., 2003b; Styler et al., 2014), parallèle au succès inédit de certaines architectures du deep-learning à la résolution de plusieurs tâches du traitement automatique du langage naturel (NLP), notamment les modèles (Bi)LSTM-CRF qui ont été prouvés de grande efficacité pour l'étiquetage du texte naturel (Akbik et al., 2018, p. 1638; Martin et al., 2020, pp. 6–7), il est devenu possible de décortiquer le problème d'étude des relations temporelles d'une manière de plus en plus simplifiée, avec des algorithmes performants et dotés d'une autonomie relativement élevée par rapport aux méthodes classiques de machine-learning. Le schéma global à deux phases : reconnaissance des objets (entités) temporels, suivie d'une classification de la relation entre eux - si elle existe – reste pratiquement le même dans les approches récentes que dans les approches classiques (Alfattni et al., 2020, p. 105; Cheng and Miyao, 2018, p. 1834).

1.3.1 La reconnaissance des objets temporels

L'identification des entités qui seraient pertinentes au sens d'une relation temporelle donnée demeure globalement inchangée. Elle est envisagée dans la pratique comme étant une sous-tâche d'étiquetage de texte, et est achevée dans la plupart des travaux récents par l'entrainement d'un modèle profond d'étiquetage de séquences (Bhatia et al., 2019, pp. 2–3; Cheng and Miyao, 2017, p. 4). L'objectif de cette première composante du système étant de prédire un ensemble

d’étiquettes convenables pour les parties découpées du texte étudié (Bhatia et al., 2019, p. 3). Selon le schéma d’annotation adopté, ces étiquettes distingueront les entités effectivement impliquées dans des relations temporelles (date, heure, durée, éventualité) des autres parties du texte. Une architecture profonde composée souvent de (Bi)LSTM, CNN, CRF, etc. (Bhatia et al., 2019; Cheng and Miyao, 2017) est entrainée en mode supervisé à cet effet. Cette composante va servir dans un premier temps à repérer et à reconnaitre les entités d’intérêt susceptibles d’agir - ou pas - à l’occasion d’une éventuelle relation temporelle (Bhatia et al., 2019, pp. 2–3; Leeuwenberg and Moens, 2018, p. 4).

Nature surfacique des annotations : La grande différence par rapport aux méthodes classiques au niveau de cette première phase provient essentiellement de la nature de plus en plus simplifiée des schémas d’annotation utilisés dans la construction des bases d’apprentissage pour de tels modèles. En effet, dans la plupart des corpus récents dédiés à l’extraction automatique des informations temporelles, la façon dont sont encodées les annotations sur les temporalités est devenue résolument surfacique (Bittar et al., 2011a, p. 3; Leeuwenberg and Moens, 2019, pp. 352–353) et n’utilise à priori pas d’informations explicites, ni sur le sens, ni sur les dépendances morphosyntaxiques, contrairement aux approches classiques. Les étiquettes dedans fournissent plutôt une normalisation des expressions qui peuvent exprimer le temps dans le texte, au lieu de s'intéresser aux constructions grammaticales sophistiquées des temporalités (Pustejovsky et al., 2003a, p. 4). Ceci a été en large partie favorisé par le développement de TimeML (Pustejovsky et al., 2003a), le formalisme originel derrière l’avènement de ce type d’annotations « surfaciques » des temporalités, qui a été plus tard standardisé sous la norme ISO-TimeML (Pustejovsky et al., 2010).

Il s’agit d’un schéma standard d'annotation pour balisage des événements, des indicateurs temporels (*les marquables*) et de leurs relations - temporelles - (*TLINKS*) dans les textes écrits en langue naturelle (Inderjeet et al., 2007, p. 1). Les objectifs de ce standard sont de fournir un modèle commun pour la création et l'utilisation d'annotations globalement surfaciques comme moyen de simplifier la gestion des informations temporelles dans les documents (Leeuwenberg and Moens, 2019, p. 353), et de permettre une catégorisation des entités et une extraction de représentations significatives pour celles-ci à base de ces métadonnées.

TimeML prévoit à la base 4 structures qui encodent ces métadonnées dans un document: deux pour les objets temporels (entités, marquables) {*EVENT, TIMEX3*}, une qui englobe trois catégories de relations {*LINK*}, et enfin une pour les marqueurs de relations {*SIGNAL*} (Pustejovsky et al., 2003a, p. 3).

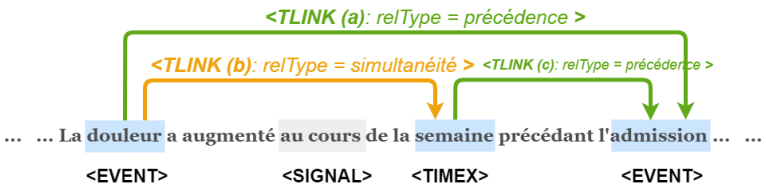

Figure 03 - Exemple d'un texte annoté par rapport au temps suivant la norme ISO-TimeML.

Le schéma TimeML initial marque les verbes, les adjectifs et les groupes nominaux avec des balises <*EVENT*>. Les expressions temporelles (dates, heures, durées) y sont marquées avec des balises <*TIMEX3*> et leurs valeurs sont normalisées, sur la base de *TIMEX3* (Inderjeet et al., 2007, p. 1). Pour l’étude stricte des temporalités, TimeML définit une balise <*TLINK*>, instance de la structure *LINK*, qui relie – d’un point de vue surfacique - un événements balisé à un autre événement (*Order TLINK*) et/ou expression temporelle (*Anchor TLINK*). La nature exacte (précédence, inclusion, simultanéité, etc.) d’une relation donnée est précisée dans un attribut *relType* (Inderjeet et al., 2007, p. 1) de la balise <*TLINK*>.TimeBank (Pustejovsky et al., 2003b) pour l’anglais, et son homologue français FrenchTimeBank (Bittar et al., 2011b) en sont des exemples célèbres de ce type de corpus annotés pour la temporalité. Thyme est un autre corpus couramment utilisé dans les applications cliniques où on retrouve encore cette tendance à alléger les annotations en limitant autant que possible l’engagement théorique. Il a été mis à disposition en anglais à l’occasion du projet THYME (Styler et al., 2014). Ce principe d’annotation superficielle permet de faciliter la construction de bases d’apprentissage suffisamment annotées sur les entités et les liens temporels entre elles, d’une manière surfacique, requérant moins d’expertise lors de l’annotation. Dans la pratique, et en plus de désambiguïser les annotations, ceci aura pour principal avantage de simplifier significativement la préparation (collection, correction, augmentation) des corpus pour le développement des modèles profonds, contrairement aux approches classiques où l’annotation délicate des entités est souvent basée sur le sens et sur des considérations induites des dépendances grammaticales complexes préétablies entre celles-ci.

1.3.2 Election, et classification des paires selon la nature de la temporalité

Les entités temporelles d'intérêt repérées – légèrement - grâce à la première composante du système forment alors des paires d'entités temporelles candidates, élues selon une stratégie donnée, et seront par la suite observées par un certain modèle classificateur. Le rôle d'un tel classificateur est de vérifier si la relation étudiée entre les entités d'une paire candidate est établie, ou pas, en observant une certaine représentation numérique (features) caractérisant ces entités, jointement avec une vérité terrain sur la nature de la relation entre elles (uniquement durant l'entrainement) (Alfattni et al., 2020, p. 7; Cheng and Miyao, 2017, p. 4; Peterson et al., 2020, p. 4; Tourille et al., 2017, p. 3).

Features automatiques : Rompant avec les anciennes méthodes, les features employées à l'entrée de cette sous-tâche de classification des temporalités et qui vont caractériser les paires d'entités candidates sont d'autant plus apprises et générées automatiquement, au lieu d'être nécessairement conçues à la main comme dans les approches classiques de machine-learning. En effet, pour résoudre des tâches similaires, les algorithmes récents du (NLP) font très fréquemment recours à un transfert des features maps incorporées dans les couches pré-entrainés de certains modèles profonds de langage, communément appelées couches d'embedding (Zhou et al., 2020, pp. 276–277), en vue d'obtenir plus tard des représentations numériques significatives – au sens d'une future tâche sous-jacente - du texte étudié à base de celles-ci. L'idée derrière l'utilisation de ces couches d'embedding pré-entrainées est de pouvoir ré-exploiter les structures latentes du texte qui ont été capturées dedans, et les adapter de nouveau aux besoins d'une tâche spécifique, la classification des relations temporelles entre ses parties dans notre cas. Cet objectif est généralement atteint par un processus de fine-tuning. La même pratique est aussi traduite par une approche d'apprentissage multi-tâches (multi-task-learning) qui, pour améliorer les performances d'une tâche souhaitée, tente d'exploiter jointement les informations apprises dans d'autres tâches parallèles connexes (Zhou et al., 2020, p. 281).

De nombreux techniques et modèles d'embedding sont de nos jours utilisées en (NLP) à des finalités similaires, tels que word2vec, Glove, FastText, ELMo, GPT, BERT, XLNet (Zhou et al., 2020, p. 282), FLAIR (Akbik et al., 2019), et BioFLAIR (Sharma and Daniel Jr, 2019). Les features calculées par projection du texte dans ces espaces sémantiques ont été prouvées de grande utilité dans la résolution de nombreuses tâches du (NLP) de par leur simplicité de construction, mais surtout grâce à leur capacité inhérente à incorporer les informations nécessaires sur les éventuelles interactions entre les entités qu'elles représentent (Ghannay et al., 2016; Zhou et al., 2020, pp. 281–282). Quelques systèmes récents dédies à l'extraction des temporalités ont essayé d'explorer ces représentations génériques selon différentes stratégies de fine-tuning, afin de caractériser numériquement la séquence d'entités dans le texte étudié avant de procéder à une tâche sous-jacente, la classification des temporalités entre elles en l'occurrence (Alfattni et al., 2020, p. 108; Peterson et al., 2020, p. 4; Tourille et al., 2017, p. 5). D'autres travaux continuent toujours de mixer ces features automatiques avec une variété de représentations classiques établies « manuellement » par des règles de dependency parsing ou autres (Alfattni et al., 2020, p. 106; Cheng and Miyao, 2017). Notons que ce choix de mixer features automatiques et manuelles rend pénible le développement de l'algorithme, relativement à la complexité des features à concevoir à la main, et moins autonome certes, notamment en mode production.

La tendance à utiliser les features automatiques rend cependant les processus de développement et de prédiction (production) entièrement automatisables. Cela va nous permettre d'éviter l'ingénierie « manuelle » et délicate des features dont nous aurons besoin à l'entrée du classificateur. Notons également que la simplification apportée par l'annotation surfacique des corpus instaurée à l'occasion de ces approches récentes facilite beaucoup le processus de développement et d'apprentissage des sous-modules du système. Elle rend en effet les relations entre les parties du texte plus faciles à caractériser, et simplifie par conséquent la construction de corpus annotés pour l'apprentissage des composantes du système, contrairement aux méthodes classiques.

2. Méthodologie

Pour résoudre le problème de repérage des entités temporelles et des éventuelles relations entre elles, nous continuons dans ce travail d'adopter le schéma fonctionnel à deux phases des approches exposées précédemment, et nous choisissons de modéliser les deux composantes du système (reconnaissance des objets temporels, classification des paires candidates) avec des architectures adaptées du deep-learning, conformément aux approches récentes explicitées ci-haut. Nous nous contentons pour la partie classification des features automatiques qui seront apprises et extraites à l'aide de certaines couches d'embedding convenablement préparées. En procédant de la sorte, nous garantissons un niveau élevé d'automatisation pour le système entier qui, une fois bien entrainé, sera complétement autonome en mode production. Cela nous évitera principalement de faire des hypothèses coûteuses lors de l'ingénierie manuelle classique des features avant la partie classification des relations. Nous essayons dans ce travail de remplacer les méthodes manuelles grammaticales de génération des features (et des étiquettes aussi), souvent utilisées dans les approches classiques, par un apprentissage automatique de ces features, et nous chercherons à savoir si de telles représentations génériques seraient utiles et suffisantes à la résolution des sous-taches envisagées

2.1 Source de données annotées : FrenchTimeBank (FTiB)

Si l'approche récente décrite dans le paragraphe (1.3) se prête bien à la résolution de la problématique soulevée, c'est en partie grâce à la disponibilité d'annotations allégées et simplifiées sur les temporalités, telles que dans TimeBank et les autres corpus qui dérivent du standard d'annotation ISO-TimeML. Durant ce travail nous nous proposons d'explorer la ressource de référence en la matière en langue française, le corpus FrenchTimeBank (FTiB).

Dans sa version actuelle (FTiB) renferme 108 exemples diversifiés de documents (paragraphes) journalistiques français dûment annotés par rapport aux temporalités, suivant le schéma standard international TimeML. A chaque document (typiquement un paragraphe) dans (FTiB) est associée une liste de plusieurs balises <*TLINK*> traduisant chacune l'existence d'une relation temporelle donnée entre deux entités du document. Le texte du document est renseigné à dans une structure *TEXT*. Les entités qui sont impliquées dans des relations temporelles y seront étiquetées par <*EVENT*> ou <*TIMEX3*>. Les relations temporelles (*TLINK)* définies sur un document peuvent être de classes différentes (simultanéité, précédence, inclusion, …), et dans (FTiB) nous en comptons 15 de ces classes. Une telle information est encodée dans l'attribut *relType* de la balise <*TLINK*> correspondante.

Deux formes de liens temporels nous intéressent dans ce corpus : les relations évènement-évènement : *Order TLINK*, et les relations évènement-temps : *Anchor TLINK*. Dans le cas d'un *Order TLINK*, deux évènements seront ordonnés dans une sens bien défini, et cette information sera précisée dans les attributs *eventInstanceId* et *relatedToEventInstance* de la balise <*TLINK*> associée au lien. Quand il s'agit plutôt d'un *Anchor TLINK*, un certain évènement sera positionné dans le temps par rapport à un certain indicateur temporel. Cela sera encore précisé dans les attributs *eventInstanceId* et *relatedToTime* du <*TLINK*> correspondant. Notons que le sens de lecture de la relation est fixe dans tous les cas ; l'évènement pointé par *eventInstanceId* est toujours positionné - au sens de *relType* - par rapport à l'évènement/indicateur temporel inscrit dans l'attribut *relatedTo* correspondant.

2.1.1 Répartition disjonctive des classes temporelles

Selon le formalisme proposé par Allen (Allen, 1983), toute relation entre deux objets temporels vus comme intervalles repose en langue naturelle sur 13 relations temporelles basiques, et peut être exprimée par composition de celles-ci. Ci-dessous nous présentons le tableau de correspondance entre les 13 relations basiques d'Allen et les 15 classes de temporalités présentes dans (FTiB). Sans perte de généralités, les relations de types « Simultaneous » et « Identity » dans (FTiB) peuvent être confondues. Idem pour « During » et « Is included » (Inderjeet et al., 2007, p. 1).

Une deuxième répartition disjonctive des relations plus simple est encore envisageable. Elle repose sur le fait que l'algèbre d'Allen est composée par : 6 relations basiques, leurs inverses, et l'identité. L'idée est de regrouper les TLINK en 7 classes disjonctives de relations temporelles. Une classe représentera donc la relation d'identité, et les 6 autres représenteront chacune une relation basique d'Allen et son inverse inversée, agrégées au sein d'une même classe temporelle (Inderjeet et al., 2007, p. 1).

Tableau 1 – Tableau de correspondance entre la base temporelle d'Allen avec 13 relations basiques, et les 15 relations dans (FTiB).

Allen : Equals	a b	FTiB : Identity (337) FTiB : Simultaneous (117)
Allen : Preceds Allen : Preceded by	a b	FTiB : Before (509) FTiB : After (184)
Allen : During Allen : Contains	a b	FTiB : During (352), Is included (365) FTiB : Includes (241)
Allen : Satrts Allen : Satrted by	a b	FTiB : Begins (40) FTiB : Begun by (50)
Allen : Finishes Allen : Finished by	a b	FTiB : Ends (23) FTiB : Ended by (52)
Allen : Overlaps Allen : Overlaped by	a b	FTiB : Overlaps before (6) FTiB : Overlaps after (4)
Allen : Meets Allen : Met by	a b	FTiB : Immediately before (16) FTiB : Immediately after (15)

2.1.2 Expression simplifiée d'un lien temporel

Nous pouvons donc modéliser tout raisonnement temporel en langue naturelle directement à l'aide des relations basiques d'Allen, ou en passant par d'autres répartitions disjonctives de celles-ci. Dans ce travail, nous adoptons la deuxième répartition des temporalités proposée ci-haut. Cette hypothèse avec peu de classes temporelles simplifiera significativement le problème en réduisant le nombre de catégories temporelles disjonctives devant être discriminées : 7 au lieu de 13 avec la répartition basique. Par hypothèse le nombre d'observations disponibles pour une classe agrégée de TLINK sera beaucoup plus élevé par rapport à la répartition basique initiale. Nous garantissons ainsi un volume plus important d'annotations par classe agrégée, ce qui facilitera par la suite le processus de développement (apprentissage) des différents modules du système.

Sans perte de généralités, nous choisissons aussi de confondre Anchor TLINK et Order TLINK. Par conséquent, nous n'aurons plus besoin de préciser dans nos étiquettes si l'entité est une expression temporelle (<TIMEX3>) ou un évènement (<EVENT>). Nous ignorons également l'information sur les marqueurs, balisés par <SIGNAL>. Les relations explicites (déclenchées par un signal) et implicites seront donc traitées de la même manière.

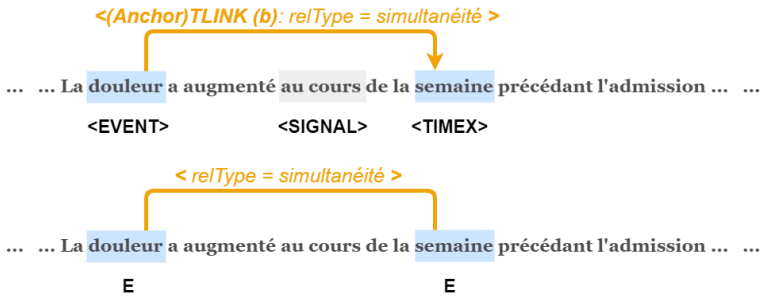

Figure 04 - Schéma montrant le passage d'un TLINK à une définition plus simplifiée du lien temporel.

Grace aux deux hypothèses explicitées ci-haut nous pouvons exprimer un lien temporel dans une phrase donnée de manière très simple en précisant uniquement les positions des deux entités qui y interviennent et évidemment la classe effective de la relation dans la répartition choisie. Une étiquette unifiée « E » balisera les entités en lien afin de distinguer celles-ci des autres parties de la phrase qui seront taguées (« O »). L'information sur la classe de la liaison peut être encodée à l'aide d'un entier c=1, 2, … 7. La donnée des positions des deux entités et la classe de la relation suffiront donc à la définition d'un lien temporel simplifié. Par exemple, l'information sur le (Anchor) $TLINK_b^s$ de la figure ci-dessus peut être simplement représenté par le triplet $([p_b^{[1][s]}, p_b^{[2][s]}]\ c_b)$, où :

- $p_b^{[1][s]}$ (Resp. $p_b^{[2][s]}$) : la position de l'objet temporel « *La douleur au genou* » (Resp. « *Trois dernières semaines* ») dans la phrase S (recevant tous les deux l'étiquette unifiée « E »).
- $c_b = i \in \{1,2,3 \ldots 7\}$, désignant la classe agrégée englobant « *Simultanéité* »

2.2 Formalisation de la tâche

Nous représentons S comme étant une séquence de tokens $t_1\ t_2\ t_3 \dots t_{Ls}$ et nous cherchons à retrouver les liens temporels distincts simplifiés – au sens du paragraphe (2.1.2) - entre les tokens de la séquence $S = t_1\ t_2\ t_3 \dots t_{Ls}$, soit :

$$\left\{ \begin{array}{c} \left(\left[p_i^{[1][s]}, p_i^{[2][s]}\right], c_i\right): \\ 1 \leq p_i^{[1][s]}, p_i^{[2][s]} \leq Ls, \\ y_{p_i^{[1][s]}}^{[tag]} = \mathrm{E} \\ y_{p_i^{[2][s]}}^{[tag]} = \mathrm{E} \\ C_i = 1,2,3 \dots 7 \end{array} \right\}_{i=1,2,3\dots B_s}$$

Avec : B_s : le nombre de liens temporels simplifiés distincts et valides rencontrés sur S, et $\forall i \in \{1,..Bs\}$: le lien i observé sur S est annoté par la donnée d'un triplet $\left(\left[p_i^{[1][s]}, p_i^{[2][s]}\right], c_i\right)$, où : $p_i^{[1][s]}$ et $p_i^{[2][s]}$ désignent les positions dans S de deux entités jugées/étiquetées temporelles ($y_{p_i^{[1][s]}}^{[tag]} = \mathrm{E}$ et $y_{p_i^{[2][s]}}^{[tag]} = \mathrm{E}$), car elles interviennent effectivement dans la relation i de classe temporelle $C_i = 1,2,3 \dots 7$.

Nous la découpons conformément à l'approche exposée brièvement ci-haut en deux sous-taches : une détection des entités censées être impliquées dans les potentielles relations temporelles, traitée comme sous-tâche d'étiquetage de texte, suivie d'une vérification de la nature des liens entre des paires celles-ci, envisagée comme sous-tâche de classification.

D'abord, notre système doit être capable de reconnaître les objets/tokens qui seraient impliqués dans des relations temporelles et les distinguer des autres parties du texte, simplement en leur attribuant l'étiquette unifiée « $\hat{E}$ » (contre « $\hat{O}$» pour le reste du texte) via un module d'étiquetage de séquences. Ceci est schématisé sur la figure suivante :

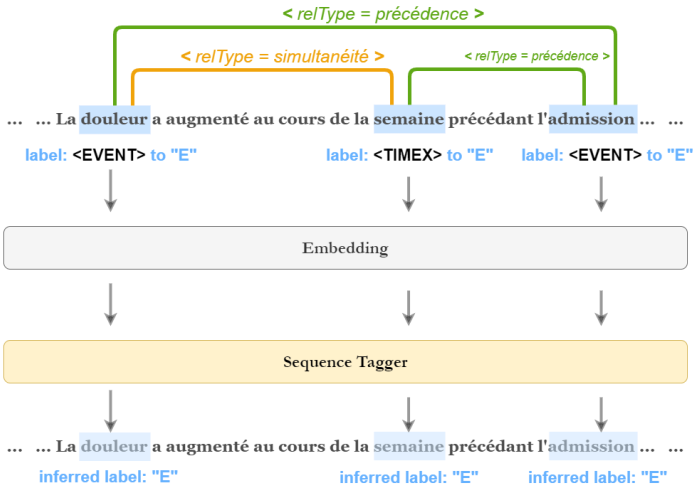

Figure 05 - Reconnaissance des entités temporelles par un module étiqueteur de séquences.

Ensuite, en procédant d'une manière brutale, nous formons l'ensemble des paires temporelles candidates, au nombre de $Q_s = C_n^2$ où n désigne le nombre d'objets temporels rencontrés sur S, obtenu par combinaisons (distinctes) des objets temporels qui étaient repérés sur S, soit :

$$\left\{ \begin{array}{c} [\hat{p}_q^{[1][s]}, \hat{p}_q^{[2][s]}]: \\ 1 \leq \hat{p}_q^{[1][s]}, \hat{p}_q^{[2][s]} \leq Ls \\ \hat{y}_{\hat{p}_q^{[1][s]}}{}^{[tag]} = \hat{E} \\ \hat{y}_{\hat{p}_q^{[2][s]}}{}^{[tag]} = \hat{E} \end{array} \right\}_{q=1,2,3\dots Q_s}$$

où les entiers $\hat{p}_q^{[1][s]}, \hat{p}_q^{[2][s]}$ désigneront pour chaque paire candidate distincte $q = 1,2,3 \dots Q_s$ les positions dans S de deux entités reconnues temporelles ($\hat{y}_{\hat{p}_q^{[1][s]}}{}^{[tag]} = \hat{E}$ et $\hat{y}_{\hat{p}_q^{[2][s]}}{}^{[tag]} = \hat{E}$) grâce aux étiquettes « $\hat{E}$ » qui leur étaient attribuées par le module étiqueteur.

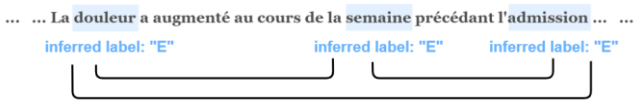

Figure 06 - Mécanisme d'élection - exhaustive - des paires candidates distinctes. Ici $Q_s = 3$.

Par stratégie d'élection brutale de l'ensemble des Q_s paires candidates, il y aura éventuellement plusieurs fausses que nous appellerons dans toute la suite les contre-exemples. Nous noterons $M_s = Q_s - B_s$ le nombre de ces fausses paires associées à S, B_s étant celui des liens valides devant figurer sur S. Présenter des échantillons de ces contre-exemples à l'entrée du classificateur devient donc nécessaire. En effet, ceci devra permettre au classificateur plus tard en mode prédiction de reconnaitre et filtrer automatiquement ces mauvaises paires forcées dans notre stratégie automatisée par recherche exhaustive des paires candidates. Notons d'ailleurs que la considération de ces fausses paires peut se traduire dans certaines phrases par une sorte de déséquilibre important entre le nombre des bonnes paires (les liens valides) et celui des contre-exemples (les faux liens). Nous devons donc tenir compte de ce déséquilibre inhérent à notre stratégie de classification lors de l'entrainement de cette partie du système. En analysant toutes ces candidates au nombre de $Q_s = M_s + B_s$, un deuxième module classificateur vérifiera alors la nature de la temporalité sur chacune d'entre elles en prédisant sa classe parmi $1, 2, \dots 7$ - si le lien devra effectivement se tenir entre les entités qui la définissent (bonne paire) - ou 0, quand la paire en question est un faux candidat (cf. contre-exemple introduit par la stratégie exhaustive d'élection).

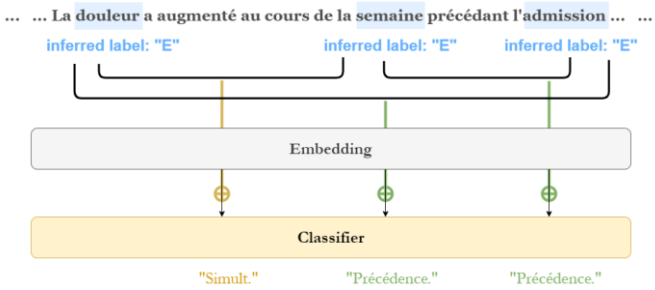

Figure 07 - Classification des paires temporelles candidates.

A l'issu de cette dernière phase, seules les paires q dont la relation était validée $\hat{C}_q(\neq 0)$ et bien reconnue ($\hat{C}_q = 1,2, \dots 7$) seront gardées, permettant ainsi de recouvrir les potentiels liens temporels simplifiés rencontrés dans S, c'est à dire :

$$\left\{ \begin{array}{c} ([\hat{p}_q^{[1][s]}, \hat{p}_q^{[2][s]}], \hat{C}_q): \\ 1 \leq \hat{p}_q^{[1][s]}, \hat{p}_q^{[2][s]} \leq Ls \\ \hat{y}_{\hat{p}_q^{[1][s]}}{}^{[tag]} = \hat{E} \\ \hat{y}_{\hat{p}_q^{[2][s]}}{}^{[tag]} = \hat{E} \\ \hat{C}_q(\neq 0) = 1,2, \dots 7 \end{array} \right\}_{q=1,2,3 \dots Q_s}$$

2.3 Architecture du système

Rappelons notre objectif que nous venons de formaliser dans le paragraphe précédent : Etant donnée une phrase $S = t_1\ t_2\ t_3 \dots t_{Ls}$, inférer :

$$\left\{ \begin{array}{c} (\left[\hat{p}_q^{[1][s]}, \hat{p}_q^{[2][s]}\right], \hat{C}_q): \\ 1 \leq \hat{p}_q^{[1][s]}, \hat{p}_q^{[2][s]} \leq Ls \\ \hat{y}_{\hat{p}_q^{[1][s]}}{}^{[tag]} = \hat{E} \\ \hat{y}_{\hat{p}_q^{[2][s]}}{}^{[tag]} = \hat{E} \\ \hat{C}_q(\neq 0) = 1,2, \dots 7 \end{array} \right\}_{q=1,2,3\dots Q_s}$$

Où , pour chaque paire distincte $q \in \{1, .. Q_s\}$: $\hat{p}_q^{[1][s]}$ et $\hat{p}_q^{[2][s]}$ désignent les positions dans S de deux entités temporelles étiquetées $\hat{E}$, et $\hat{C}_q \in \{0\} \cup \{1, \dots\}$: la classe effective de la paire (du lien) les reliant. $Q_s = M_s + B_s = C_n^2$ étant le nombre des paires candidates obtenues par combinaisons distinctes des n objets temporels inférés sur S, B_s celui des bonnes paires (liens valides, $\hat{C}_q \neq 0$) et M_s celui des contre-exemples ($\hat{C}_q = 0$) .

2.3.1 Couches externes d'embedding

Nous commençons d'abord par projeter la séquence de tokens $t_1 : t_{Ls}$ dans certains espaces sémantiques via une agrégation de quelques couches d'embedding convenablement pré-entrainées. Nous tirons parti des structures de langage apprises automatiquement dans ces couches afin de générer une première représentation numérique vectorielle significative de la séquence de tokens du texte étudié. Dans ce travail, nous allons explorer plusieurs techniques d'embedding, statiques (Glove, FastText) et dynamiques ou contextualisées (Flair), et des combinaisons de celles-ci, afin de garantir une appréhension enrichie des contextes et de leurs spécificités lors de la représentation du texte. Nous aboutissons ainsi à une première séquence $E_1^{extern} : E_{Ls}^{extern} = \theta_{extern}(t_1 : t_{Ls})$ de vecteurs génériques représentatifs du texte étudié qui seraient plus tard utilisées à la résolution des tâches sous-jacentes.

2.3.2 Couche spécifique d'embedding

Les couches d'embedding importées ne seront pas ré-entrainées dans notre système pour maintes raisons ; les dimensions de ces couches sont généralement grandes et imposées quand importées d'un modèle pré-entrainé externe, et seront probablement mal adaptées aux dimensions internes de nos sous-modules, notamment quand ces dernières doivent être petites. Pour compenser cela dans la représentation du texte avant de passer à une tâche sous-jacente, nous choisissons d'introduire une couche spécifique d'embedding dit contextualisé dont les paramètres $\theta_{specific}$ seront cette fois entrainables. Cette couche sera en effet entrainée à l'occasion des sous-taches envisagées, permettant ainsi d'obtenir une deuxième séquence $E_1^{specific} : E_{Ls}^{specific} = \theta_{specific}(t_1 : t_{Ls})$ de représentations pour les tokens du texte qui seraient par construction ajustables en fonction de nos objectifs spécifiques d'entrainement, contrairement aux embeddings importés, et qui vont probablement les consolider.

2.3.3 Concaténation des différentes couches d'embedding

Par la suite, nous concaténons ces différentes représentations et nous caractérisons la phrase $S = t_1 : t_{Ls}$ par une seule séquence $E_1^{Total} : E_{Ls}^{Total}$ de vecteurs embedding, où : $\forall j \in \{1, .. Ls\}$: $E_j^{Total} = \left[E_j^{extern}, E_j^{specific}\right]$.

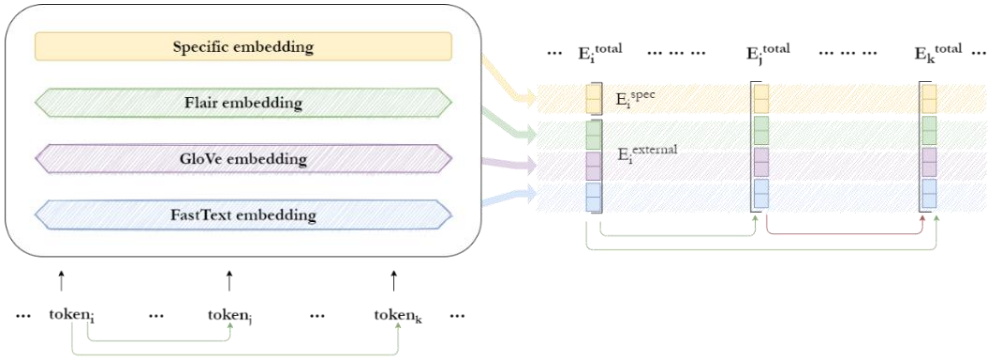

Figure 08 - Projection d'une séquence de tokens via les couches d'embedding et concaténation des features.

Cette séquence de features constituera dans toute la suite l'entrée effective à chaque sous-module responsable d'une sous-tâche donnée.

2.3.4 (bi)LSTM-CRF pour la reconnaissance des entités temporelles

En premier lieu, la séquence de features $E_1^{Total} : E_{Ls}^{Total}$ sera fournie en entrée à une cellule (bi)LSTM. Son rôle est de calculer une séquence de scores intermédiaires ; ses états cachés $h_1 : h_{Ls} = \theta_{LSTM}(E_1^{Total} : E_{Ls}^{Total})$. Un module CRF utilisera ensuite ces scores pour inférer une séquence finale d'étiquettes $\hat{y}_1^{[tag]} : \hat{y}_{Ls}^{[tag]} = \theta_{CRF}(h_1 : h_{Ls})$.

De telles étiquettes vont finalement permettre de reconnaître les tokens potentiellement impliqués dans des relations temporelles ($\hat{y}_j^{[tag]} = \hat{E}$) et les distinguer des autres tokens ($\hat{y}_j^{[tag]} = \hat{O}$) de la phrase S. L'entrainement de cette composante sera effectué en mode supervisé, en observant les vérités terrain $y_1^{[tag]} : y_{Ls}^{[tag]}$ sur l'exemple de phrase S, où : $\forall j \in \{1, .. Ls\}$: $y_j^{[tag]} \in \{\mathrm{O}, \mathrm{E}\}$, traduisant l'implication ($y_j^{[tag]} = E$) , ou non ($y_j^{[tag]} = O$), du token j dans un certain lien temporel observé sur la séquence S.

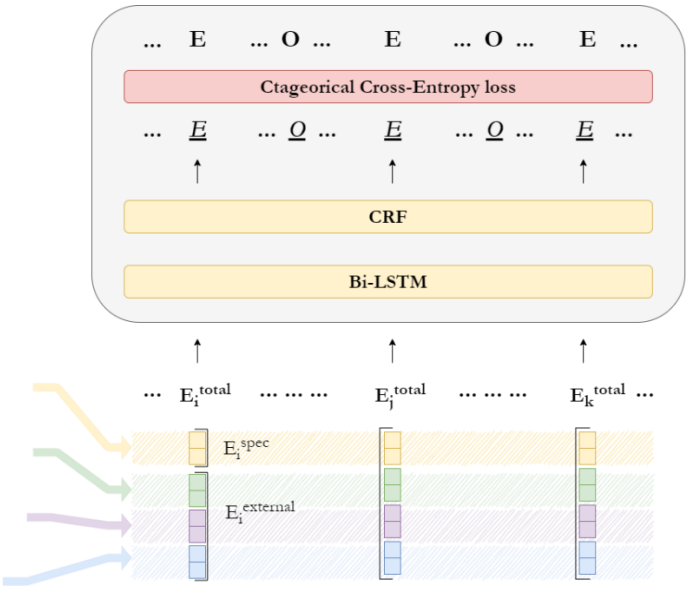

Figure 09 - Etiquetage des tokens temporels par un module (bi)LSTM-CRF.

2.3.5 Vérification des liens temporels par classification des paires candidates

A l'issu de la phase précédente, toute paire candidate distincte $q \in \{1, 2, .., Q_s = M_s + B_s\}$ de tokens étiquetés par (« Ê ») sera examinée par un classificateur dont le rôle est de prédire la classe $\hat{C}_q$ du lien et vérifier s'il doit effectivement se tenir. Notons $\hat{p}_q^{[1][S]}$ et $\hat{p}_q^{[2][S]}$ les positions dans S des deux entités qui définissent une telle paire candidate, i.e. : $\hat{y}^{[tag]}{}_{\hat{p}_q^{[1][S]}} = \hat{E}$ et $\hat{y}^{[tag]}{}_{\hat{p}_q^{[2][S]}} = \hat{E}$, grâce à la composante précédente.

Pour obtenir une seule représentation vectorielle de la paire q que nous noterons E_q^{clf}, nous choisissons tout simplement de concaténer les features génériques $E_{\hat{p}_q^{[1][S]}}^{Total}$ et $E_{\hat{p}_q^{[2][S]}}^{Total}$ qui se trouvent aux positions $\hat{p}_q^{[1][S]}$ et $\hat{p}_q^{[2][S]}$ de S, soit :

$$E_q^{clf} = \left[E_{\hat{p}_q^{[1][S]}}^{Total}, E_{\hat{p}_q^{[2][S]}}^{Total}\right].$$

Notons que le mécanisme d'obtention d'un tel vecteur peut être entièrement automatisé car il repose uniquement sur les structures apprises dans les couches d'embedding utilisées, d'où l'intérêt à ce type de features automatiques dans notre approche. Le rôle du module classificateur sera alors de prédire la classe $\hat{C}_q = \theta_{clf}\left(E_q^{clf}\right) \in \{0\} \cup \{1, ..\}$ de la relation

temporelle entre les entités de la paire étudiée en observant la représentation vectorielle générique E_q^{clf} que nous venons de construire pour elles.

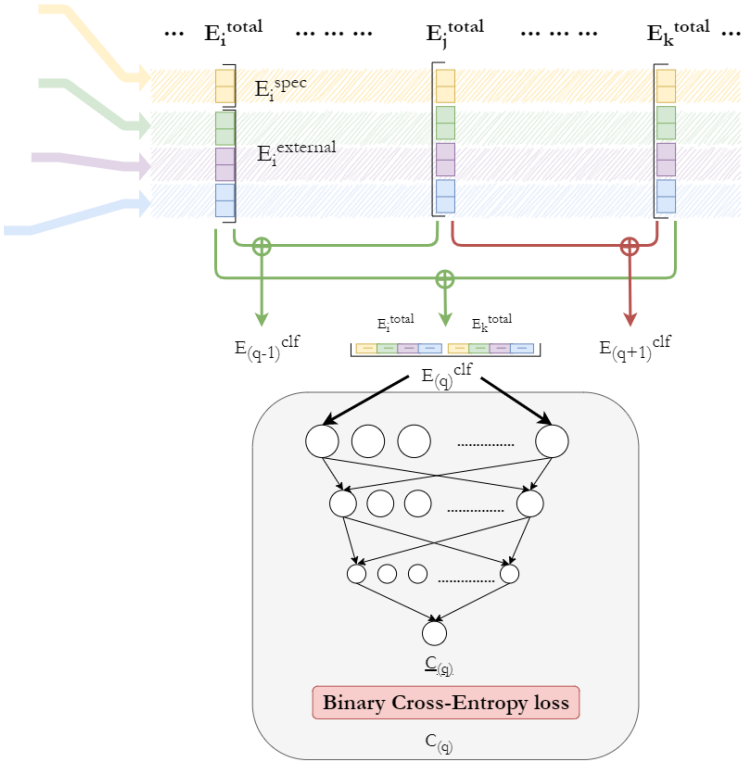

Figure 10 - Module de classification des paires candidates selon la nature exacte de la temporalité.

Typiquement, l'ensemble des paires candidates doit être en partie constitué de $B_s = Q_s - M_s$ bonnes paires correspondants à des liens valides que nous cherchons à retrouver, et pour lesquelles le classificateur devra prédire une valeur de $\hat{C}_q \neq 0$ correspondant à la classe exacte du lien parmi $\{1, 2, .. 7\}$ (cf. liens valides représentés en vert sur la figure ci-dessus). Le reste ($M_s = Q_s - B_s$) seront des fausses paires dont la présence est due à notre stratégie brutale automatisée d'élection.

Le classificateur doit aussi être capable de distinguer ces fausses candidates par une valeur $\hat{C}_q = 0$ car le lien présumé devrait être invalide et serait un faux lien (cf. contre-exemple représenté en rouge sur la figure ci-haut).

Le recouvrement des liens temporels simplifiés rencontrés sur S reviendrait donc à filtrer l'ensemble inféré de toutes les paires candidates, pour ne garder que celles dont les liens sont validés $\hat{C}_q(\neq 0)$ et bien reconnus ($\hat{C}_q = 1,2, \dots 7$) par le classificateur, i.e. :

$$\left\{ \begin{array}{c} \left(\left[\hat{p}_q^{[1][s]}, \hat{p}_q^{[2][s]}\right], \hat{C}_q\right): \\ 1 \leq \hat{p}_q^{[1][s]}, \hat{p}_q^{[2][s]} \leq Ls \\ \hat{y}^{[tag]}{}_{\hat{p}_q^{[1][s]}} = \hat{E}, \\ \hat{y}^{[tag]}{}_{\hat{p}_q^{[2][s]}} = \hat{E}, \\ \hat{C}_q(\neq 0) = 1,2, \dots 7 \end{array} \right\}_{q=1,2,3 \dots Q_s}$$

2.4 Schémas d'entrainement

Deux schémas sont envisageables pour entrainer les différentes composantes de notre système. Rappelons son fonctionnement à deux phases, et que selon notre formalisation la résolution de la tâche a été divisée en deux sous-tâches : reconnaissance des entités temporelles, puis classification des paires candidates selon la nature de la temporalité.

2.4.1 Approche par apprentissages séparés

Une première façon de faire consiste à entrainer d'abord le module (bi)LSTM-CRF pour la sous-tâche étiquetage du texte $S = t_1 : t_{Ls}$ avec comme fonction de coût :

$$Loss_{[tag]}((y_1^{[tag]}, .., y_{Ls}^{[tag]}), (\hat{y}_1{}^{[tag]}, .., \hat{y}_{Ls}{}^{[tag]})) = \sum_{p=1}^{Ls} [- \sum_{c} y_{p=c}^{[tag]} . \log(\hat{y}_{p=c}{}^{[tag]})]$$

Où : $y_1^{[tag]} : y_{Ls}^{[tag]}$ désigne la séquence des vérités terrain « E / O » attribuées initialement à la séquence de tokens de la phrase S en fonction des liens effectifs qui y sont rencontrés, $\hat{y}_1{}^{[tag]} : \hat{y}_{Ls}{}^{[tag]} = \theta_{CRF}(\theta_{LSTM}(E_1^{Total} : E_{Ls}^{Total}))$ représente celle des étiquettes « $\hat{E}/\hat{O}$ » inférées par le module (bi)LSTM-CRF pour la même séquence, et où c couvre l'ensemble des étiquettes distinctes que le module devra inférer pour chaque token du texte, dans notre cas : $\{\hat{E}, \hat{O}\}$ ($\cup \{ < unk >, < start >, < end >\}$ optionnellement). Les poids qui vont être ajustés à l'occasion de cette première phase d'étiquetage sont $\{\theta_{specific}, \theta_{LSTM}, \theta_{CRF}\}$.

Puis dans un second temps, entrainer séparément un module classificateur à base des features $E_1^{Total} : E_{Ls}^{Total}$, obtenues par projection de S via les couches d'embeddings $\{\theta_{extern}, \theta_{specific}\}$ qui viennent d'être utilisées et entrainées dans le module (bi)LSTM-CRF, pour classer les paires candidates selon la nature éventuelle de la temporalité. Sur la phrase S, un ensemble de $B_s + M_S$ paires candidates sera rencontré par le classificateur (B_s liens valides, $M_s$contre-exemples). Pour toute paire candidate $q \in \{1, .. B_S + M_S\}$ le classificateur va donc inférer une classe $\hat{C}_q = \theta_{clf}(E_q^{clf}) \in \{0\} \cup \{1, ...\}$ traduisant sa classe exacte, ou le fait que c'est un contre-exemple (notamment quand $\hat{C}_q = 0$). Ce classificateur aura comme fonction de coût :

$$\begin{aligned} Loss_{[clf]}([(C_1^{[m]}, .., C_{M_s}^{[m]}), (\hat{C}_1^{[m]}, .., \hat{C}_{M_s}^{[m]})], [(C_1^{[b]}, .., C_{B_s}^{[b]}), (\hat{C}_1^{[b]}, .., \hat{C}_{B_s}^{[b]})]) \\ = \alpha_{B_s} . \sum_{q=1}^{B_s} -[C_q^{[b]} . \log(\hat{C}_q^{[b]}) + (1 - C_q^{[b]}) . \log(1 - \hat{C}_q^{[b]})] \\ + \alpha_{M_s} . \sum_{q=1}^{M_s} -[C_q^{[m]} . \log(\hat{C}_q^{[m]}) + (1 - C_q^{[m]}) . \log(1 - \hat{C}_q^{[m]})] \end{aligned}$$

B_s étant le nombre des (liens) paires valides devant être rencontrés sur S, et M_s désignant celui des contre-exemples introduits par méthode d'élection exhaustive. L'entrainement de cette composante se fera en mode supervisé en observant des vérités terrain sur des liens valides $(C_1^{[b]}, .., C_{B_s}^{[b]})$ et des contre-exemples $(C_1^{[m]}, .., C_{M_s}^{[m]})$ pré-annotés de S. Les poids qui seraient entrainées à l'occasion de cette deuxième phase d'optimisation séparée sont ceux du classificateur, i.e. $\{\theta_{clf}\}$. Les hyper-paramètres α_{B_s} *et* α_{M_s} seront fixés manuellement en fonction des données d'entrainement et vont servir principalement à contourner les difficultés liées au problème de déséquilibre bons (liens) vs contre-exemples

Notons toutefois que pour cette approche d'entrainement séparée, où l'apprentissage (les optimisations) des deux sous-tâches (des deux coûts) va se dérouler séparément, les features génériques E_q^{clf} utilisées à l'entrée du classificateur vont être par construction peu informatives à l'égard de la tâche de classification. Rappelons qu'une telle représentation sera obtenue par concaténation des features $E_{\hat{p}_q^{[1][S]}}^{Total}$ et $E_{\hat{p}_q^{[2][S]}}^{Total}$ qui se trouvent aux positions $\hat{p}_q^{[1][S]}$ et $\hat{p}_q^{[2][S]}$ de la séquence, et que celles-ci vont dépendre des couches d'embedding $\{\theta_{extern}, \theta_{specific}\}$. Seule la couche $\theta_{specific}$ est en fait autorisée à changer de forme, et selon cette approche séparée d'entrainement $\theta_{specific}$ sera uniquement bien réajustée pour incorporer les structures nécessaires à étiqueter les potentielles entités temporelles et les distinguer des autres tokens du texte, apprises à l'occasion de la sous-tâche d'étiquetage, mais elle va toujours ignorer si un éventuel lien de la classe étudiée devrait se tenir, car aucun chemin n'est encore prévu pour relier ses poids à la fonction de coût du classificateur.

2.4.2 Approche multi-task-learning

Pour combler ce manque d'information sur la partie classification au niveau des features génériques que nous devons introduire dans le classificateur, nous choisissons d'entrainer tout le système selon une approche multi-task-learning où nous optimiserons simultanément les deux sous-tâches via un seul objectif d'entrainement combiné :

$$Loss_{[mtl]} \coloneqq \alpha_{mtl}.\log\left(Loss_{[tag]}\right) + \beta_{mtl}.\log(Loss_{[clf]})$$

Le chemin de back-propagation des gradients à partir de cet objectif d'entrainement combiné créera cette fois deux sources d'informations servant à mettre à jour les poids de la couche spécifique d'embedding $\theta_{specific}$, d'une part celles sur les étiquettes temporelles, et d'autre part celles sur la classe des potentiels liens. Les structures apprises dedans doivent capter par construction ces deux niveaux d'information et seraient probablement beaucoup plus informatives comme features, notamment à l'égard de la classification d'où l'intérêt de cette approche multi-task-learning.

Les hyper-paramètres $\alpha_{mtl}, \beta_{mtl}$, et le recours à la log ont pour objectif de ramener les deux coûts en une même échelle et éviter qu'une tâche l'emportera sur l'autre durant l'optimisation, auquel cas une tâche parmi les deux aura moins d'intérêt et serait pratiquement ignorée dans le processus d'optimisation jointe. Le choix de ces hyper-paramètres doit être fait avec attention et dépendra fortement des données d'entrainement.

2.5 Classification multiple ou binaire

Dans cette partie nous exposons et commentons les deux options qui s'offrent à nous concernant la modélisation du résultat attendu de la partie classification des paires candidates de notre système : devons-nous traiter d'une manière séparée chaque classe (classification binaire) ou traiter toutes les classes à la fois (classification multiple) ? Dans le premier cas, seul module de classification se chargera typiquement de classer des liens temporels provenant des différentes classes (classification multiple des paires), alors que dans le deuxième, un modèle sera consacré au repérage de chaque classe temporelle en mode binaire en prédisant si le lien présumé devra se tenir ou non (classification binaire des paires). Nous allons tenir compte éventuellement des spécificités des données (FTiB).

Le tableau ci-après résume le nombre d'exemples d'annotations qu'on pourra retrouver par classe agrégée de liens temporels dans (FTiB) dans sa version actuelle. Nous pouvons déjà y observer un déséquilibre important entre les occurrences des différentes classes temporelles. Certains liens temporels tels que « immédiatement avant, chevauche avant » sont rarement rencontrés sur les documents dont nous disposons. En contrepartie, l'expression d'autres types de temporalités comme « inclusion, simultanéité, précédence » est très récurrente et est beaucoup plus susceptible d'avoir lieu sur ces mêmes documents.

Tableau 2 - Répartition des TLINK dans (FTiB) selon les différentes classes temporelles.

Classe	***(1)*** $\{\boldsymbol{Includes}\} \cup \{\boldsymbol{During}\}^{-1} \cup \{\boldsymbol{Is\ included}\}^{-1}$	***(2)*** $\{\boldsymbol{Before}\} \cup \{\boldsymbol{After}\}^{-1}$	***(3)*** $\{\boldsymbol{Identity}\} \cup \{\boldsymbol{Simult.}\}$	***(4)*** $\{\boldsymbol{Begins}\} \cup \{\boldsymbol{Begun\ by}\}^{-1}$	***(5)*** $\{\boldsymbol{Ends}\} \cup \{\boldsymbol{Ended\ by}\}^{-1}$	***(6)*** $\{\boldsymbol{Imm.before}\} \cup \{\boldsymbol{Imm.after}\}^{-1}$	***(7)*** $\{\boldsymbol{Overlaps\ before}\} \cup \{\boldsymbol{Overlaps\ after}\}^{-1}$
#TLINK	**852**	**479**	**391**	79	49	31	10

Un classificateur multi-classes entrainé sur des exemples de ces données à discriminer en une seule fois toutes les temporalités du texte sera donc facilement confus face à ce déséquilibre, et ses décisions seraient par conséquent probablement biaisées au détriment des classes minoritaires.

Par ailleurs, pour assurer le bon déroulement de la partie classification, surtout quand le nombre de classes à discriminer est important, il faut veiller à que les représentations utilisées à l'entrée du classificateur soient indépendantes d'une classe à une autre. Autrement dit, les features utilisées afin de caractériser une paire candidate de classe donnée doivent être indépendants de celles utilisés pour les paires provenant des autres classes temporelles.

Selon notre approche, pour une paire donnée ces features sont obtenues automatiquement par concaténation des vecteurs embedding correspondants aux entités qui la définissent. Quand par exemple deux paires de classes différents doivent

partager une entité en commun, ce qui est souvent le cas, ce problème d'indépendance apparaît ; en fait dans ce cas une partie du vecteur utilisé pour caractériser une paire sera intégralement répliquée dans celui caractérisant la deuxième paire, à cause de l'entité en commun entre les deux, introduisant ainsi une sorte de dépendance entre les deux.

Pour contourner ces difficultés liées principalement au nombre multiple de classes à discriminer, nous choisissons d'entrainer la composante classification de notre système en mode binaire. Dans notre travail un modèle sera donc consacré à chaque classe de temporalités dans la répartition choisie, et le rôle de chaque modèle serait de prédire pour une paire candidate q d'objets temporels si un lien de type la classe étudiée devrait se tenir ($\hat{C}_q = 1$), ou pas ($\hat{C}_q = 0$). Ainsi nous simplifions significativement le problème de classification. L'agrégation des prédictions des différents modèles (pour les différentes classes) sur un même document doit permettre finalement le repérage et la reconstruction de toutes les classes temporelles.

2.6 Métriques d'évaluation

Les deux sous-tâches impliquées dans notre modélisation sont en réalité deux tâches de classification. D'abord, la reconnaissance des objets temporels d'un échantillon de texte S (étiquetés E) qui se fait en prédisant la classe correcte d'une étiquette pour chaque token de la séquence S parmi $\{\hat{O}, \hat{E}\}$. Ensuite, la reconstruction des potentiels liens temporels entre ces objets temporels, par classification - binaire - de certaines paires candidates ; $\hat{C}_q = \theta_{clf}\left(E_q^{clf}\right) \in \{0\} \cup \{1, \dots\}$, $q = 1,2 \dots Q_s$. Il convient alors de mesurer les performances des deux composantes de notre système avec des matrices de confusion, une matrice par sous-tâche. Le choix de la matrice de confusion va nous permettre de résumer les performances d'un certain modèle de classification et à partir de ses valeurs nous pouvons ensuite calculer de nombreuses métriques d'évaluation telles que : le rappel, la précision, ou encore le score f1 pour les différentes classes.

Tableau 3 - Matrice de confusion permettant de mesurer les performances de la reconnaissance des objets temporels.

	Etiquette inférée : Ê	***Etiquette inférée : Ô***
Etiquette réelle : E	**TE :** le nombre de tokens initialement annotés (E) et encore prédits comme Ê	**FO :** le nombre de tokens initialement annotés (E) mais inversés en Ô
Etiquette réelle : O	**FE :** le nombre de tokens initialement annotés (O) mais inversés en Ê	**TO :** le nombre de tokens initialement annotés (O) et encore prédits comme Ô

Pour la partie étiquetage du texte, les étiquettes $\{\hat{E}, \hat{O}\}$ inférées par le module (bi)LSTM-CRF pour un échantillon de texte pré-annoté S peuvent être évaluées contre les étiquettes réelles observés sur l'échantillon comme le montre la matrice de confusion ci-dessus. Ensuite, nous pouvons calculer le rappel, la précision et le f1 score pour chaque classe d'étiquettes comme suit:

$$\text{Precision}_{(\text{E})} = \frac{\text{TE}}{(\text{TE} + \text{FE})}, \quad \text{Recall}_{(\text{E})} = \frac{\text{TE}}{(\text{TE} + \text{FO})}, \quad \text{f1}_{(\text{E})} = 2 \cdot \frac{\text{Precision}_{(\text{E})} \cdot \text{Recall}_{(\text{E})}}{\text{Precision}_{(\text{E})} + \text{Recall}_{(\text{E})}}$$

$$\text{Precision}_{(\text{O})} = \frac{\text{TO}}{(\text{TO} + \text{FO})}, \quad \text{Recall}_{(\text{O})} = \frac{\text{TO}}{(\text{TO} + \text{FE})}, \quad \text{f1}_{(\text{O})} = 2 \cdot \frac{\text{Precision}_{(\text{O})} \cdot \text{Recall}_{(\text{O})}}{\text{Precision}_{(\text{O})} + \text{Recall}_{(\text{O})}}$$

Pour la partie classification des paires temporelles candidates observées sur S, également envisagée dans ce travail comme une sous-tâche de classification binaire, nous pouvons construire similairement la matrice de confusion ci-après :

Tableau 4 - Matrice de confusion permettant de mesurer les performances de la classification des paires candidates.

	Classe inférée : $\widehat{\mathbf{1}}$ ***(Lien valide)***	***Classe inférée :*** $\widehat{\mathbf{0}}$ ***(Contre-exemple)***
Classe réelle : 1 (Lien valide)	**TP :** le nombre de paires initialement classées (1) et encore classées comme $\widehat{\mathbf{1}}$	**FN :** le nombre de paires initialement classées (1) mais inversées en $\widehat{\mathbf{0}}$
Classe réelle : 0 (Contre-exemple)	**FP :** le nombre de paires initialement classées (0) mais inversés en $\widehat{\mathbf{1}}$	**TN :** le nombre de paires initialement classées (0) et encore classées comme $\widehat{\mathbf{0}}$

Et calculer ensuite toutes les métriques (précision, recall, f1) qui en découlent, idem à la tâche précédente, et qui vont nous aider cette fois à juger la qualité de la classification des paires candidates de S:

$$\text{Precision}_{(\text{P})} = \frac{\text{TP}}{(\text{TP} + \text{FP})}, \qquad \text{Recall}_{(\text{P})} = \frac{\text{TP}}{(\text{TP} + \text{FN})}, \qquad \text{f1}_{(\text{P})} = 2 \cdot \frac{\text{Precision}_{(\text{P})} \,.\, \text{Recall}_{(\text{P})}}{\text{Precision}_{(\text{P})} + \text{Recall}_{(\text{P})}}$$

$$\text{Precision}_{(\text{N})} = \frac{\text{TN}}{(\text{TN} + \text{FN})}, \qquad \text{Recall}_{(\text{O})} = \frac{\text{TN}}{(\text{TN} + \text{FP})}, \qquad \text{f1}_{(\text{N})} = 2 \cdot \frac{\text{Precision}_{(\text{N})} \,.\, \text{Recall}_{(\text{N})}}{\text{Precision}_{(\text{N})} + \text{Recall}_{(\text{N})}}$$

Notons que la définition de ces métriques, présentées jusqu'ici sur un seul échantillon S, reste valable pour un ensemble d'échantillons de test $\{S\}_{test}$. Il suffit simplement d'accumuler dans une même matrice les statistiques calculées d'un échantillon à un autre dans $\{S\}_{test}$ et sommer toutes ces valeurs pour aboutir à une seule matrice évaluant les résultats de classification dans tout l'ensemble d'échantillons de test.

2.7 Ré-échantillonnage des exemples annotées de (FTiB)

Rappelons que (FTiB) renferme uniquement 108 documents annotés par rapport aux différentes classes temporelles. Si nous choisissons d'entrainer notre système avec ces documents tels qu'ils sont, c'est-à-dire sans ré-échantillonnage ni pré-exploitation du texte et des annotations, nous aurons pour chaque modèle (un modèle par classe) au plus 108 exemples annotés d'entrainement, ce qui serait largement insuffisant pour le développement et l'évaluation des architectures profondes utilisées.

Afin de remédier partiellement à ce problème, nous proposons une technique de ré-échantillonnage des documents initiaux de (FTiB) qui va nous permettre d'exploiter au maximum les annotations disponibles dedans, mais surtout d'augmenter le nombre d'exemples d'entrainement.

L'idée consiste simplement à associer une fenêtre à chaque lien temporel dans le document initial et considérer la partie du document (et ses annotations) délimitée par cette fenêtre comme un nouvel échantillon d'entrainement. La largeur d'une telle fenêtre sera déterminée par les positions des entités qui définissent le lien en question. Pour garantir une meilleure appréhension du contexte de la fenêtre nous élargissons son étendu avec un paramètre $(\pm Delta)$ à gauche et à droite. La partie du texte couverte par la fenêtre élargie, et tous les liens temporels qui y apparaissent, constitueront ainsi un nouvel exemple annoté d'entrainement. Autrement, si dans un document initial de (FTiB) nous rencontrons d liens temporels, nous allons en extraire d exemples distincts d'entrainement.

Ce mécanisme est explicité dans la figure suivante où nous exhibons un exemple abstrait de document contenant trois liens temporels valides représentés en vert : (1), (2) et (3). A partir de ce document, nous pouvons donc extraire trois exemples de phrases annotées, délimitée chacune par un contexte propre à elle.

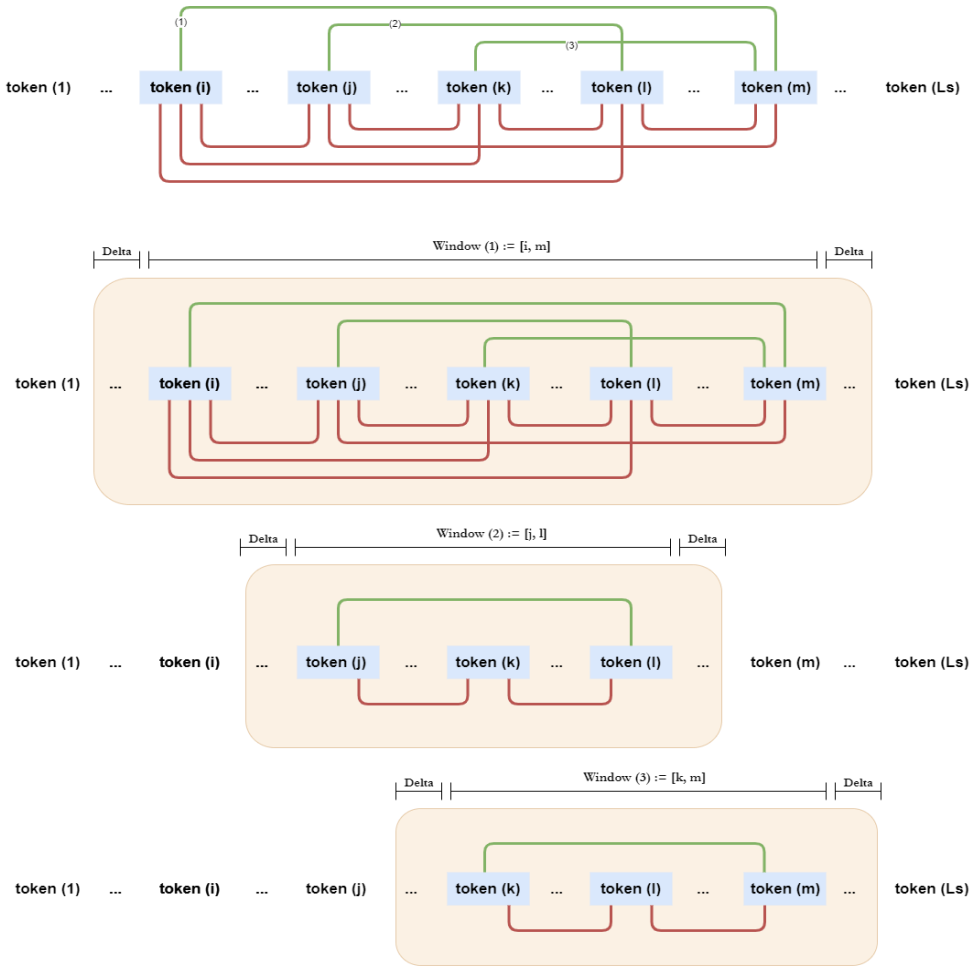

Figure 11 - Mécanisme de ré-échantillonnage des documents annotés de (FTiB)

En procédant de la sorte, nous garantissons d'une part le fait que tous les liens annotés dans (FTiB) soient présentés au modèle, chacun au moins une fois, et d'autre part nous augmentons le nombre d'échantillons de phrases annotées utilisables durant l'entrainement. Notons également que cette approche contribue partiellement à la réduction systématique du taux de déséquilibre entre bons et contre-exemples rencontrés sur une phrase et mentionné plus haut dans le paragraphe (2.2) (cf. deux dernières fenêtres de la figure, contre l'annotation initiale).

Nous choisissons finalement de ramener dans des limites plus étroites les fenêtres obtenues par la méthode de pré-exploitation des données annotées décrite ci-haut en ignorant celles renfermant plus que U_{max} liens temporels, ou pour lesquelles le quotient $\frac{nombre\ "O"}{nombre\ "E"}$ dépassera une valeur maximale V_{max} autorisée pour le déséquilibre entre le nombre des tokens étiquetés "E" et celui des tokens étiquetés "O" dans la fenêtre.

Quand U_{max} est faible, avoir au plus U_{max} liens temporels sur un exemple de fenêtre d'entrainement revient à dire qu'un nombre faible d'entités temporelles y sera présent. Ceci doit être d'intérêt pour notre système dans le sens qu'il permettra de réduire systématiquement le niveau de déséquilibre bons vs contre-exemples inhérent à la construction de notre méthode de classification. Le nombre de contre-exemples devant être introduits sera en fait plus faible. C'est l'objectif derrière la première hypothèse de filtrage explicitée précédemment.

Cependant, réduire le nombre d'entités rencontrées sur la phrase, quoiqu'adapté à la phase classification, peut avoir un impact négatif sur la partie étiquetage du texte. En fait, cette situation favorisera l'apparition cette fois d'un déséquilibre entre les tokens étiquetés « E » (peu nombreux) et ceux étiquetés « O » qui seraient probablement beaucoup plus nombreux, notamment quand la fenêtre est très large ou d'une manière équivalente quand le lien délimitant la fenêtre prend lieu entre deux entités très éloignées. La deuxième hypothèse essaye de contourner cet impact en veillant à que le déséquilibre soit relativement acceptables dans nos données d'apprentissage et ne dépasse pas V_{max} comme valeur.

3. Expériences

3.1 Construction des ensembles d'apprentissage

Dans la suite de ce travail nous construisons nos bases d'apprentissages pour les différentes classes temporelles à partir des données de (FTiB) en suivant la démarche du paragraphe (2.7), et en se fixant 15 et 5 comme valeurs pour U_{max} et V_{max} respectivement. C'est-à-dire, dans nos bases d'entrainement, un exemple de phrase S présentera au plus $U_{max} = 15$ liens temporels de classe donnée définis le long de la phrase, et il y aura au plus $V_{max} = 5$ fois plus de « O » que de « E » sur cette phrase. Ces valeurs ont été fixées après plusieurs essais expérimentaux.

Il est également important à ce stade de souligner la difficulté d'adapter notre façon de modéliser le problème au cas d'une entité temporelle s'étalant sur deux ou plusieurs tokens. En effet, quand une telle entité multi-tokens est impliqué dans une relation temporelle donnée, notre construction des features pour la paire en question, obtenue jusqu'ici par simple concaténation des deux vecteurs embeddings pour les deux tokens repérés de la paire, ne tient plus, et perd tout son sens. Le traitement de ce type d'évènements temporels multi-tokens souvent rencontrés dans les textes (cf. groupes nominaux) demeure toutefois incontournable et nécessiterait d'autres considérations, et pour la sous-tâche d'étiquetage (changer complétement le système d'étiquetage) et pour la sous-tâche de classification (notamment la manière d'agréger les features). Nous filtrons dans la suite nos bases d'entrainement de ces exemples de liens pour ne garder que les liens temporels qui impliquent des entités mono-token. La répartition des échantillons de textes annotés finalement obtenus est démontrée sur la figure suivante :

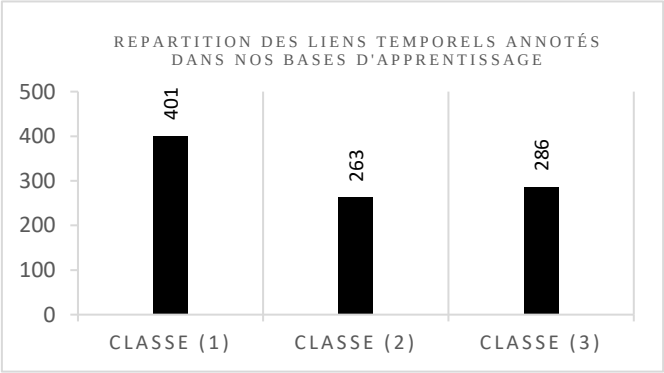

Figure 12 - Nombre d'échantillons d'apprentissage extraits par classe temporelle retenue.

Selon la répartition des classes temporelles qu'on a choisie dans (2.1), nous devons idéalement entrainer 7 modèles distincts, mais vu le nombre très limité d'observations disponibles pour la majorité des classes dans (FTiB), nous choisissons conduire le reste de l'étude uniquement sur les 3 classes qui apparaissent sur l'histogramme de la figure 12. Notons cependant que le volume de données annotées ainsi obtenues pour les différentes classes temporelles reste très insuffisant pour un entrainement efficace des architectures profondes impliquées dans notre système, et ne nous permettra par conséquent pas de bien mesurer le pouvoir généralisateur de ces modules.

L'objectif de la suite de ce travail est d'étudier sous ces conditions expérimentales très spécifiques l'utilité des features génériques utilisées dans notre système à résoudre les deux sous-tâches d'étiquetage et de classification des liens temporels. Une instance de notre système sera typiquement implémentée et évaluée pour chaque classe temporelle de la figure 12. Dans chacune de ces instances, nous allons essayer plusieurs combinaisons de couches d'embedding de natures différentes : statique comme (Glove, FastText), ou contextualisée comme (Flair). Notre objectif serait à chaque fois de tester la capacité des structures apprises dans ces représentations génériques concaténées du texte à modéliser les liens temporels de la classe étudiée ; reconnaitre les objets temporels dans un premier temps, et classer les paires candidates après.

3.2 Résultats d'apprentissage

Pour chaque classe temporelle parmi les 3 que nous souhaitons étudier, nous entrainons 8 instances distinctes de notre système qui diffèrent par la combinaison des couches externes d'embedding (parmi Glove, FastText, Flair) utilisées dedans afin de construire des features génériques. Chacune de ces instances va être entrainer selon les deux modes d'entrainement exposés dans le paragraphe (2.4). Nous allons ensuite évaluer les performances du système dans chaque scénario {classe temporelle donnée, combinaison donnée d'embedding, schéma d'entrainement donné} en se référant aux métriques introduites dans le paragraphe (2.6).

Pour la comparabilité des résultats, nous choisissons d'entrainer tous les modèles externes d'embedding $\{\theta_{Glove}, \theta_{FastText}, \theta_{Flair}\}$ (pour Glove, FastText, Flair respectivement) qui vont être utilisés dans nos systèmes sur un même corpus non-annoté de textes journalistiques, de contexte proche à celui des documents dans (FTiB), construit à partir de diverses sources (wikipédia, web-crawling, magazines, etc.). La couche spécifique d'embedding $\theta_{specific}$, rappelons-le, serait quant à elle entrainée dans les sous-taches envisagées avec des échantillons annotés de texte. Toutes ces couches auront aussi la même dimension et auront ainsi le même niveau de contribution dans la génération des features automatiques quand agrégées ensemble.

Les hyper-paramètres (profondeurs, dimensions, $\alpha_{mtl}, \beta_{mtl}, \alpha_B, \alpha_M$) des différentes composantes du système ont été choisis manuellement après plusieurs essais sur le volume réduit de données d'apprentissage dont nous disposons, et seront fixes dans toutes les futures démonstrations par soucis de comparabilité.

Les tableaux (5), (6) et (7) de l'annexe ci-après résument les performances des différents systèmes correspondants aux différentes configurations d'entrainement décrites ci-dessus.

Nous y constatons que pour toutes les classes temporelles étudiées, peu importe le mode d'entrainement du système, la sous-tâche de reconnaissance des entités temporelles avec le module (bi)LSTM-CRF s'achève relativement avec de bons résultats au sens de la matrice de confusion représentée dans le Tableau 3, calculée à chaque fois (pour chaque classe) sur un ensemble test d'échantillons fixé. Les performances de cette sous-tâche d'étiquetage de texte sont résumées dans les tableaux par les scores f1 calculés pour les deux catégories d'étiquettes $\{E, O\}$ sur l'ensemble de test correspondant. Nous remarquons que ces scores sont globalement d'autant plus élevés que l'on concatène des couches externes d'embedding parmi $\{\theta_{Glove}, \theta_{FastText}, \theta_{Flair}\}$ avec la couche spécifique d'embedding $\theta_{specific}$, et nous en déduisons alors l'efficacité de l'agrégation des features génériques obtenues par projections sur ces couches à la résolution de cette première sous-tâche d'étiquetage des objets temporels. Notons que la concaténation de ces différentes couches d'embedding va modifier la complexité du système dans chaque scénario; concaténer plusieurs couches d'embedding augmentera certes cette complexité et impactera dès lors la qualité du processus d'apprentissage (système nécessiterait un volume beaucoup plus élevé de données).

Par ailleurs, nous constatons que la sous-tâche de classification des paires candidates (lien valide vs contre-exemple) donne des résultats globalement moins bons pour toutes les classes étudiées, au sens des métriques d'évaluation présentées dans le Tableau 4, et ce pratiquement pour les deux modes d'entrainement. Les résultats étant particulièrement plus stables en mode par apprentissages séparés. Ces résultats sont attendus et doivent en fait être impactés par les circonstances particulières d'entrainement (volume très limité de données), et par la délicatesse inhérente à la partie classification des paires candidates dans notre modélisation (nature des features utilisées, stratégie de concaténation des objets temporels pour former les paires candidates, stratégie brutale d'élection des paires candidates, déséquilibre bons vs contre-exemples, définition et paramètres du coût de classification et du coût combiné). Nous remarquons cependant que l'agrégation de certaines couches d'embeddings parmi $\{\theta_{Glove}, \theta_{FastText}, \theta_{Flair}\}$ permet toujours de construire des features qui contribuent à des prédictions moyennement justes et relativement plus bonnes que celles obtenues uniquement avec $\theta_{specific}$ afin de caractériser les paires candidates et les classer, notamment en approche séparée d'entrainement.

Pour la classe temporelle (1) qui englobe les liens de type $\{Includes, During, Is\ included\}$ (cf. Tableau 5), les meilleurs performances en mode d'apprentissage multi-task-learning (resp. séparé) ont été enregistrées pour la combinaison $\{\theta_{specific}\} \cup \{\theta_{glove}, \theta_{FastText}, \theta_{Flair}\}$ (resp. $\{\theta_{specific}\} \cup \{\theta_{Flair}\}$) de couches d'embedding. Les plus meilleures performances pour la classe temporelle (2) qui englobe les liens de type $\{Before,\ After\}$ (cf. Tableau 6) ont été quant à elles obtenues pour la combinaison $\{\theta_{specific}\} \cup \{\theta_{FastText}\ \theta_{Flair}\}$ (resp. $\{\theta_{specific}\} \cup \{\theta_{Flair}\}$) de couches d'embedding, en mode multi-task-learning (resp. séparé). Et finalement, pour la classe temporelle (3) qui renferme les liens de type $\{Identity, Simultaneous\}$ (cf. Tableau 7), nous avons enregistré une meilleure performance avec la combinaison $\{\theta_{specific}\} \cup \{\theta_{FastText}\ \theta_{Flair}\}$ (resp. $\{\theta_{specific}\} \cup \{\theta_{Glove}\ \theta_{Flair}\}$) en mode multi-task-learning (resp. séparé).

Conclusion

Nous venons de voir dans le cadre de partie expérimentale du rapport que le fait de concaténer les représentations génériques issues d'une certaine combinaison de couches d'embedding convenablement pré-entrainées était d'efficacité variable à la résolution des deux sous-taches envisagées dans notre système de repérage des liens temporels. Ces représentations agrégées ont donné de bonnes performances au sens des métriques définies dans le paragraphe (2.6) quand utilisées à l'entrée du sous-module (bi)LSTM-CRF et ont permis dans un premier temps de bien de repérer les entités temporelles, mais elles étaient relativement moins utiles après comme entrées au module classificateur des paires candidates, et ceci est dû principalement à la complexité relative à la modélisation toute la phase classification dans notre travail. Dans les deux cas, nous avons pu observer que l'agrégation d'une ou plusieurs couches externes parmi $\{\theta_{Glove}, \theta_{FastText}, \theta_{Flair}\}$ avec $\theta_{specific}$ améliorait significativement les deux sous-tâches. Les expériences réalisées ne permettent cependant pas de généraliser, mais servent comme proof-of-concept de l'utilité de ces features automatiques et démontrent que la tâche de repérage et de classification des temporalités peut dorénavant reposer sur ces features dont la génération est automatique, avec d'autres considérations certes. Les performances limites des architectures profondes utilisées à l'occasion des différentes composantes de notre système devraient réellement dépendre du volume et de la variabilité des données pré-annotées qui seraient utilisées lors de l'apprentissage. Les performances finales des différentes sous-tâches de classification envisagées peuvent être mesurées comme dans notre travail en termes de matrices de confusion et toutes les autres métriques qui en dérivent (rappel, précision, f1). Notons aussi que la qualité finale des résultats définie par le pouvoir généralisateur du système sur des textes d'un autre domaine (autres contextes) dépendra de l'aspect des annotations et des paires fournies durant l'entrainement ; de quelle manière sommes-nous en train de définir la relation temporelle surfacique et les étiquettes correspondantes ? Ce qui reviendrait à clarifier quelles relations mériteraient d'être annotées, et comment les annoter.

Perspectives

Les expériences réalisées dans notre travail peuvent être considérées comme simple preuve de l'utilité des features génériques pour résoudre d'une manière automatisée le problème de repérage des temporalités divisé en deux sous-tâches, mais elles souffrent concrètement d'un manque important de données d'entrainement, pratiquement pour toutes les classes temporelles étudiées, et ceci est principalement dû au volume très limité de documents initialement disponibles dans (FTiB). Pour un entrainement efficace des modules profonds utilisées dans notre modélisation, il faut idéalement augmenter le volume des données d'apprentissage, et ceci est désormais réalisable sans beaucoup d'effort de par la nature relativement « allégée » des schémas d'annotation utilisés. Les exigences d'annotation manuelle sont devenues plus faciles à satisfaire et permettent donc une construction plus rapide et plus facilitée des corpus d'apprentissage. Ayant accès à un volume plus important de données pré-annotées, et en partant d'un système optimal entrainé sur ces données au sens des métriques proposées, nous pouvons continuer d'augmenter activement le corpus de départ par un cycle semi-automatisé d'active-learning, assisté à chaque fois par correction manuelle des annotations prédites sur un ensemble de nouveaux exemples non-annotés. Pour contourner la difficulté d'adapter notre modélisation à deux phases au cas des entités temporelles multi-tokens, devant s'étaler sur deux ou plusieurs tokens adjacents, évoquée brièvement dans le paragraphe (3.1) et due principalement au choix du schéma d'étiquetage $\{E, O\}$ utilisé jusqu'ici, rappelons-le : E pour les entités temporelles s'étalant sur un seul token, et O pour tous les autres tokens, nous pouvons changer complétement le schéma d'étiquetage et introduire une troisième étiquette I qui identifiera les parties intermédiaires d'une entité temporelles multi-tokens (comme le schéma BIO). Le premier token d'une telle entité peut être toujours marqué par E pour signaler son début, et les parties (tokens) adjacentes à ce premier token, qui forment la suite de l'entité, recevront alors l'étiquette I. Nous devons par la suite adapter systématiquement notre façon de représenter une paire temporelle candidate, faite jusqu'ici par simple concaténation des deux vecteurs features de chaque entité mono-token de la relation. Nous pouvons en effet encoder l'ensemble des features d'une entité multi-token (ou mono-token) grâce à une cellule à mémoire (RNN ou autres) et aboutir finalement à une seule représentation par entité, même si l'entité est multi-token, puis concaténer les vecteurs des deux entités comme précédemment, et procéder à la classification. Notons finalement l'importance de revoir et améliorer la stratégie d'élection exhaustive des paires candidates qui contribue jusqu'ici systématiquement à la création d'une sorte de déséquilibre intrinsèque du au mécanisme de construction des contre-exemples. Nous pouvons à cet effet élire plutôt les paires d'objets temporels candidates en fonction des chemins de dépendances entre ces objets (par part-of-speech-tagging par exemple). La redéfinition du coût de classification, et du coût combiné lors de l'apprentissage multi-task peut aussi contribuer à contourner l'effet indésirable d'un tel déséquilibre, ce qui revient dans notre modélisation actuelle à faire un bon choix pour les hyper-paramètres α_{mtl}, $\beta_{mtl}, \alpha_B, \alpha_M$, et à revoir la définition du coût de classification.

Annexe

Tableau 5 - Performances des systèmes entrainées pour la classe (1) des temporalités. Les lignes MTL désignent les résultats en mode d'entrainement multi-task-learning, et SEP désignent ceux du mode par apprentissages séparées. La combinaison d'embedding (et les scores correspondants) permettant d'obtenir les meilleures performances en chaque mode a été repérée en gras sur le tableau.

Classe (1)		*Specific*	*Specific + Glove*	*Specific + FastText*	***Specific + Flair***	*Specific + Glove + FastText*	*Specific + Glove + Flair*	*Specific + FastText + Flair*	***Specific + Glove + FastText + Flair***
Etiquetage F1 score #O : 586 #E : 189	*MTL*	*O : 92.62%* *E : 75%*	*O : 94.84%* *E : 83.19%*	*O : 95.94%* *E : 86.74%*	*O : 95.51%* *E : 85.74%*	*O : 91.93%* *E : 71.72%*	*O : 96.27%* *E : 87.97%*	*O : 95.46%* *E : 84.83%*	***O : 96.34%*** ***E : 88.34 %***
	SEP	O : 93.23% E : 80.39%	O :94.62% E :85.88%	-------- --------	**O : 96.43%** **E :88.89%**	---------- ----------	O: 95.86% E: 83.78%	---------- ----------	---------- ----------
Classification Valeurs sur la diagonale de la matrice de confusion	*MTL*	*Bons : 0%* *Faux : 100%*	*Bons : 51%* *Faux : 46%*	*Bons : 9.4%* *Faux : 91%*	*Bons : 0%* *Faux : 100%*	*Bons : 0%* *Faux : 100%*	*Bons : 96%* *Faux : 5.5%*	*Bons : 9.4%* *Faux : 89%*	***Bons : 52%*** ***Faux : 45%***
	SEP	Bons : 58% Faux : 67%	Bons :81% Faux :55%	---------- ----------	**Bons : 69%** **Faux : 64%**	---------- ----------	Bons :79% Faux :52%	--------- ---------	---------- ----------

Tableau 6 - Performances des systèmes entrainées pour la classe (2) des temporalités. La combinaison d'embedding (et les scores correspondants) permettant d'obtenir les meilleures performances en chaque mode a été repérée en gras sur le tableau.

Classe (2)		*Specific*	*Specific + Glove*	*Specific + FastText*	***Specific + Flair***	*Specific + Glove + FastText*	*Specific + Glove + Flair*	***Specific + FastText + Flair***	*Specific + Glove + FastText + Flair*
Etiquetage F1 score #O : 586 #E : 189	*MTL*	*O : 87.66%* *E : 27.95%*	*O : 92.03%* *E : 70.14%*	*O : 94.09%* *E : 80.00%*	*O : 93.38%* *E : 72.13%*	*O : 93.27%* *E : 77.24%*	*O : 94.82%* *E : 80.59%*	***O : 94.37%*** ***E : 79.71%***	*O : 93.71%* *E : 78.32 %*
	SEP	O : 82.53% E : 71.64%	O :91.77% E :84.06%	-------- --------	**O : 84.26%** **E :73.24%**	---------- ----------	O: 83.39% E: 71.64%	---------- ----------	---------- ----------
Classification Valeurs sur la diagonale de la matrice de confusion	*MTL*	*Bons : 94%* *Faux : 5.9%*	*Bons : 51%* *Faux : 47%*	*Bons : 5.7%* *Faux : 91%*	*Bons : 100%* *Faux : 0%*	*Bons : 97%* *Faux : 2.9%*	*Bons : 66%* *Faux : 29%*	***Bons : 45%*** ***Faux : 82%***	*Bons : 37%* *Faux : 59%*
	SEP	Bons : 46% Faux : 50%	Bons :63% Faux :25%	---------- ----------	**Bons : 57%** **Faux : 75%**	---------- ----------	Bons :72% Faux :25%	--------- ---------	---------- ----------

Tableau 7 - Performances des systèmes entrainées pour la classe (3) des temporalités. La combinaison d'embedding (et les scores correspondants) permettant d'obtenir les meilleures performances en chaque mode a été repérée en gras sur le tableau.

Classe (3)		*Specific*	*Specific + Glove*	*Specific + FastText*	*Specific + Flair*	*Specific + Glove + FastText*	***Specific + Glove + Flair***	***Specific + FastText + Flair***	*Specific + Glove + FastText + Flair*
Etiquetage F1 score #O : 586 #E : 189	*MTL*	*O : 91.63%* *E : 57.53%*	*O : 91.15%* *E : 76.74%*	*O : 94.64%* *E : 86.36%*	*O : 95.34%* *E : 87.86%*	*O : 94.69%* *E : 86.04%*	*O : 92.68%* *E : 80.92%*	***O : 95.28%*** ***E : 88.26%***	*O : 94.85%* *E : 87.00 %*
	SEP	O : 78.55% E : 60.61%	O :84.29% E :83.87%	-------- --------	O : 77.68% E :68.18%	---------- ----------	**O: 92.48%** **E: 85.33%**	---------- ----------	---------- ----------
Classification Valeurs sur la diagonale de la matrice de confusion	*MTL*	*Bons : 0%* *Faux : 100%*	*Bons : 20%* *Faux : 76%*	*Bons : 63%* *Faux : 38%*	*Bons : 9.8%* *Faux : 90%*	*Bons : 80%* *Faux : 14%*	*Bons : 98%* *Faux : 2%*	***Bons : 53%*** ***Faux : 44%***	*Bons : 16%* *Faux : 84%*
	SEP	Bons : 33% Faux : 58%	Bons :50% Faux :53%	---------- ----------	Bons : 33% Faux : 64%	---------- ----------	**Bons :33%** **Faux :81%**	--------- ---------	---------- ----------

Références

Akbik, A., Bergmann, T., Vollgraf, R., 2019. Pooled Contextualized Embeddings for Named Entity Recognition, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). Presented at the NAACL-HLT 2019, Association for Computational Linguistics, Minneapolis, Minnesota, pp. 724–728. https://doi.org/10.18653/v1/N19-1078

Akbik, A., Blythe, D., Vollgraf, R., 2018. Contextual String Embeddings for Sequence Labeling, in: Proceedings of the 27th International Conference on Computational Linguistics. Presented at the COLING 2018, Association for Computational Linguistics, Santa Fe, New Mexico, USA, pp. 1638–1649.

Alfattni, G., Peek, N., Nenadic, G., 2020. Extraction of temporal relations from clinical free text: A systematic review of current approaches. J. Biomed. Inform. 108, 103488. https://doi.org/10.1016/j.jbi.2020.103488

Allen, J.F., 1983. Maintaining knowledge about temporal intervals. Commun. ACM 26, 832–843. https://doi.org/10.1145/182.358434

Bhatia, P., Celikkaya, B., Khalilia, M., Senthivel, S., 2019. Comprehend Medical: A Named Entity Recognition and Relationship Extraction Web Service, in: 2019 18th IEEE International Conference On Machine Learning And Applications (ICMLA). Presented at the 2019 18th IEEE International Conference On Machine Learning And Applications (ICMLA), pp. 1844–1851. https://doi.org/10.1109/ICMLA.2019.00297

Bittar, A., Amsili, P., Denis, P., 2011a. French TimeBank : un corpus de référence sur la temporalité en français. Presented at the TALN 2011 - Traitement Automatique des Langues Naturelles, Laboratoire d'Informatique de Robotique et de Microélectronique, p. 259.

Bittar, A., Danlos, L., Amsili, P., Denis, P., 2011b. Building a TimeBank for French: a reference Corpus Annotated According to the ISO-TimeML Standard.

Callahan, A., Fries, J.A., Ré, C., Huddleston, J.I., Giori, N.J., Delp, S., Shah, N.H., 2019. Medical device surveillance with electronic health records. NPJ Digit. Med. 2, 94. https://doi.org/10.1038/s41746-019-0168-z

Cheng, F., Miyao, Y., 2018. Inducing Temporal Relations from Time Anchor Annotation, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers). Presented at the NAACL-HLT 2018, Association for Computational Linguistics, New Orleans, Louisiana, pp. 1833–1843. https://doi.org/10.18653/v1/N18-1166

Cheng, F., Miyao, Y., 2017. Classifying Temporal Relations by Bidirectional LSTM over Dependency Paths. pp. 1–6. https://doi.org/10.18653/v1/P17-2001

Ghannay, S., Favre, B., Estève, Y., Camelin, N., 2016. Word Embedding Evaluation and Combination, in: Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC'16). Presented at the LREC 2016, European Language Resources Association (ELRA), Portorož, Slovenia, pp. 300–305.

Inderjeet, M., Ben, W., Marc Verhagen, 2007. Three approaches to learning tlinks in timeml.

Jason, A.F., Ethan, S., Saelig, K., Scott, L.F., Jose, P., Alison, C., Nigam, H.S., 2017. Ontology-driven weak supervision for clinical entity classification in electronic health records | Nature Communications [WWW Document]. URL https://www.nature.com/articles/s41467-021-22328-4 (accessed 6.23.21).

Kolomiyets, O., Moens, M.-F., 2011. A survey on question answering technology from an information retrieval perspective. Inf. Sci. 181, 5412–5434. https://doi.org/10.1016/j.ins.2011.07.047

Leeuwenberg, A., Moens, M.-F., 2019. A Survey on Temporal Reasoning for Temporal Information Extraction from Text. J. Artif. Intell. Res. 66, 341–380. https://doi.org/10.1613/jair.1.11727

Leeuwenberg, A., Moens, M.-F., 2018. Temporal Information Extraction by Predicting Relative Time-lines. ArXiv180809401 Cs.

Leeuwenberg, A., Moens, M.-F., 2017. Structured Learning for Temporal Relation Extraction from Clinical Records, in: Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers. Presented at the EACL 2017, Association for Computational Linguistics, Valencia, Spain, pp. 1150–1158.

Martin, L., Muller, B., Suárez, P.J.O., Dupont, Y., Romary, L., de la Clergerie, É.V., Seddah, D., Sagot, B., 2020. CamemBERT: a Tasty French Language Model. Proc. 58th Annu. Meet. Assoc. Comput. Linguist. 7203–7219. https://doi.org/10.18653/v1/2020.acl-main.645

Mirza, P., Tonelli, S., 2016. CATENA: CAusal and TEmporal relation extraction from NAtural language texts, in: Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers. Presented at the COLING 2016, The COLING 2016 Organizing Committee, Osaka, Japan, pp. 64–75.

Peterson, K., Jiang, G., Liu, H., 2020. A Corpus-Driven Standardization Framework for Encoding Clinical Problems with HL7 FHIR. J. Biomed. Inform. 110, 103541. https://doi.org/10.1016/j.jbi.2020.103541

Pustejovsky, J., Castaño, J., Ingria, R., Saurí, R., Gaizauskas, R., Setzer, A., Katz, G., 2003a. TimeML: A Specification Language for Temporal and Event Expressions.

Pustejovsky, J., Hanks, P., Saurí, R., See, A., Gaizauskas, R., Setzer, A., Radev, D., Sundheim, B., Day, D., Ferro, L., Lazo, M., 2003b. The TimeBank corpus. Proc. Corpus Linguist.

Pustejovsky, J., Lee, K., Bunt, H., Romary, L., 2010. ISO-TimeML: An International Standard for Semantic Annotation, in: Proceedings of the Seventh International Conference on Language Resources and Evaluation (LREC'10). Presented at the LREC 2010, European Language Resources Association (ELRA), Valletta, Malta.

Sharma, S., Daniel Jr, R., 2019. BioFLAIR: Pretrained Pooled Contextualized Embeddings for Biomedical Sequence Labeling Tasks. ArXiv190805760 Cs.

Styler, W.F., IV, Bethard, S., Finan, S., Palmer, M., Pradhan, S., de Groen, P.C., Erickson, B., Miller, T., Lin, C., Savova, G., Pustejovsky, J., 2014. Temporal Annotation in the Clinical Domain. Trans. Assoc. Comput. Linguist. 2, 143–154. https://doi.org/10.1162/tacl_a_00172

Tang, B., Wu, Y., Jiang, M., Chen, Y., Denny, J.C., Xu, H., 2013. A hybrid system for temporal information extraction from clinical text. J. Am. Med. Inform. Assoc. JAMIA 20, 828–835. https://doi.org/10.1136/amiajnl-2013-001635

Tourille, J., Ferret, O., Tannier, X., Névéol, A., 2017. Temporal information extraction from clinical text, in: Conference of the European Chapter of the Association for Computational Linguistics. Valence, Spain.

Zhou, M., Duan, N., Liu, S., Shum, H.-Y., 2020. Progress in Neural NLP: Modeling, Learning, and Reasoning. Engineering 6, 275–290. https://doi.org/10.1016/j.eng.2019.12.014